A PROJECT REPORT ON

CLOUD-BASED MULTI-AGENT AI ORCHESTRATION PLATFORM

SUBMITTED TO

MIT SCHOOL OF COMPUTING LONI PUNE

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

BY

Tanishq Sachin Sonawane ADT24SOCBD132

Keyur Arvind Mhatre ADT24SOCBD062

Varun Hitendra Patil ADT24SOCBD139

TEAM CLOUD CLASH GROUP BCCC41

Under the guidance of

Prof. Vrushali Kondhalkar



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MIT SCHOOL OF COMPUTING

MIT Art Design and Technology University

Rajbaug Campus Loni Kalbhor Pune 412201

2026–27

MIT SCHOOL OF COMPUTING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MIT ART DESIGN AND TECHNOLOGY UNIVERSITY
RAJBAUG CAMPUS LONI KALBHOR PUNE 412201

CERTIFICATE

This is to certify that the project report entitled

CLOUD-BASED MULTI-AGENT AI ORCHESTRATION PLATFORM

Submitted by

Tanishq Sachin Sonawane ADT24SOCBD132

Keyur Arvind Mhatre ADT24SOCBD062

Varun Hitendra Patil ADT24SOCBD139

is a bonafide work carried out by them under the supervision of Prof. Vrushali Kondhalkar and is submitted towards the partial fulfillment of the requirements of MIT ADT University Pune for the award of the degree of Bachelor of Technology in Computer Science and Engineering during the academic year 2026–27.

Prof. Vrushali Kondhalkar
Guide

_____
Head of Department

_____
Chief Coordinator PBL

_____
Dean

Seal or Stamp of the College

Place: Pune
Date: _____

DECLARATION

We the team members

Name	Enrollment number
Tanishq Sachin Sonawane	ADT24SOCBD132
Keyur Arvind Mhatre	ADT24SOCBD062
Varun Hitendra Patil	ADT24SOCBD139

hereby declare that the project work incorporated in the present report entitled CLOUD-BASED MULTI-AGENT AI ORCHESTRATION PLATFORM is original work. This work in part or in full has not been submitted to any University for the award of a degree or a diploma. We have properly acknowledged material collected from secondary sources wherever required. We solely own responsibility for the originality of the entire content.

Date: _____

Name and Signature of the Team Members

Tanishq Sachin Sonawane ADT24SOCBD132
Signature: _____

Keyur Arvind Mhatre ADT24SOCBD062
Signature: _____

Varun Hitendra Patil ADT24SOCBD139
Signature: _____

Name and Signature of Guide

Prof. Vrushali Kondhalkar
Signature: _____

Seal or Stamp of the College
Place: Pune
Date: _____

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MIT SCHOOL OF COMPUTING
RAJBAUG LONI KALBHOR PUNE 412201

EXAMINER'S APPROVAL CERTIFICATE

The project report entitled CLOUD-BASED MULTI-AGENT AI ORCHESTRATION PLATFORM submitted by Tanishq Sachin Sonawane (ADT24SOCBD132), Keyur Arvind Mhatre (ADT24SOCBD062), and Varun Hitendra Patil (ADT24SOCBD139) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering during the academic year 2026–27 of MIT ADT University MIT School of Computing Pune is hereby approved.

Examiners

1. Name: _____
   Signature: _____

2. Name: _____
   Signature: _____

Place: Pune

Date: _____

Seal or Stamp of the College

ACKNOWLEDGEMENT

We express our gratitude to our project guide Prof. Vrushali Kondhalkar for academic guidance and supervision of Cloud-Based Multi-Agent AI Orchestration Platform. We appreciate the opportunity to develop and explain the project within the Project Based Learning framework at MIT School of Computing MIT ADT University.

We thank the Department of Computer Science and Engineering for the academic setting in which this work is presented. The project connects software architecture, databases, cloud deployment, web development, and artificial intelligence through a common orchestration problem.

We acknowledge the authors of the research papers, the maintainers of the open-source tools, and the technical documentation referenced in this report. Their work provides the foundations for tool-enabled agents, collaborative workflows, embedded persistence, and browser event streaming. Sources are identified in the bibliography and cited where relevant.

We also acknowledge the contributions of our team members in preparing the project for review. We remain responsible for the technical explanations and for verifying the implementation and results submitted for assessment.

Tanishq Sachin Sonawane
ADT24SOCBD132

Keyur Arvind Mhatre
ADT24SOCBD062

Varun Hitendra Patil
ADT24SOCBD139

ABSTRACT

Cloud-Based Multi-Agent AI Orchestration Platform addresses the coordination required to turn individual AI assistants into a persistent team-based service. The project implementation AgentOps connects specialized provider-backed agents to Telegram, WhatsApp, Discord, a command-line interface, and a web dashboard. Users can direct a request to an agent or a team leader and inspect the associated execution state.

The platform uses a TypeScript and Node.js daemon with a SQLite-backed durable queue. Per-agent serialization prevents overlapping turns for the same agent while independent agents may run concurrently. Provider adapters launch existing agentic command-line harnesses and normalize their streamed events. A bracket-tag communication protocol creates delegated work with root and thread lineage. Markdown memory, session mapping, schedules, and heartbeat prompts support continuity beyond one interactive conversation.

The report presents the system concepts, architecture, requirements, deployment boundary, and evaluation methodology. Documented controls include authentication, sender pairing, attachment path validation, bounded retries, dead-letter inspection, and recovery of interrupted work. The current implementation is a single-host self-hosted design that can be placed on a suitable cloud virtual machine; distributed multi-node execution remains future work.

The available project material establishes an engineering description rather than measured performance evidence. Accordingly the report identifies expected outputs and proposed tests without inventing benchmark results. Key limitations include duplicate external effects after retry provider dependence message amplification and concentrated host availability. The project provides a concrete basis for studying observable and recoverable multi-agent assistance.

Keywords: Multi-agent systems; AI orchestration; durable queue; persistent memory; cloud deployment; agent collaboration.

CONTENTS

Certificate I
Declaration II
Examiner's Approval Certificate III
Acknowledgement IV
Abstract V
Contents VI
List of Figures VIII
List of Tables IX
Nomenclature X
CHAPTER 1 INTRODUCTION 1
1.1 PROJECT OVERVIEW 1
1.2 PROBLEM STATEMENT 2
1.3 MOTIVATION AND OBJECTIVES 3
1.4 SCOPE AND APPLICATION SCENARIOS 4
1.5 REPORT ORGANIZATION AND CONTRIBUTIONS 5
CHAPTER 2 CONCEPTS AND METHODS 6
2.1 AGENTS AND SPECIALIZED ROLES 6
2.2 LAYERED CLIENT SERVER ARCHITECTURE 7
2.3 DURABLE QUEUE AND PROCESSING LIFECYCLE 8
2.4 TEAM ROUTING AND COMMUNICATION 9
2.5 MEMORY SKILLS AND CONTEXT CONTINUITY 10
2.6 EVENTS SCHEDULING AND SUPERVISION 11
2.7 TECHNOLOGY SELECTION AND DEPLOYMENT 12
CHAPTER 3 LITERATURE SURVEY 13
3.1 TOOL ENABLED REASONING AND SYSTEM CONTEXT 13
3.2 AUTOGEN AND CONVERSATIONAL COLLABORATION 14
3.3 METAGPT AND STRUCTURED ROLE WORKFLOWS 15
3.4 INFRASTRUCTURE FOUNDATIONS AND RESEARCH GAP 16

CONTENTS CONTINUED

CHAPTER 4 PROJECT PLAN 17
4.1 DEVELOPMENT METHODOLOGY 17
4.2 WORK BREAKDOWN AND DELIVERABLES 18
4.3 RISK MANAGEMENT AND DEPENDENCIES 19
4.4 ACCEPTANCE MILESTONES AND DOCUMENTATION 20
CHAPTER 5 SOFTWARE REQUIREMENT SPECIFICATION 21
5.1 PRODUCT SCOPE AND SYSTEM CONTEXT 21
5.2 USER CLASSES AND USE CASES 22
5.3 FUNCTIONAL REQUIREMENTS FOR EXECUTION 23
5.4 FUNCTIONAL REQUIREMENTS FOR COLLABORATION 24
5.5 DATA REQUIREMENTS AND LOGICAL MODEL 25
5.6 EXTERNAL INTERFACES AND API CONTRACT 26
5.7 NONFUNCTIONAL AND SECURITY REQUIREMENTS 27
5.8 HARDWARE SOFTWARE AND OPERATING CONSTRAINTS 28
CHAPTER 6 RESULTS 29
6.1 DOCUMENTED IMPLEMENTATION OUTCOMES 29
6.2 EXPECTED DIRECT REQUEST DEMONSTRATION 30
6.3 EXPECTED TEAM COLLABORATION DEMONSTRATION 31
6.4 RELIABILITY AND RECOVERY OBSERVATIONS 32
6.5 DISCUSSION OF RESULTS AND LIMITATIONS 33
CHAPTER 7 SOFTWARE TESTING 34
7.1 TEST STRATEGY AND ENVIRONMENT 34
7.2 FUNCTIONAL TEST CASES 35
7.3 SECURITY AND FAILURE TEST CASES 36
7.4 PERFORMANCE AND QUALITY EVALUATION 37
7.5 TRACEABILITY AND RELEASE ACCEPTANCE 38
CHAPTER 8 CONCLUSION AND FUTURE WORK 39
8.1 CONCLUSION 39
8.2 FUTURE WORK 40
Bibliography 41
Annexure A Publications and Research Paper 42
Annexure B Plagiarism Report 48
Annexure C Empathy Chart and Solution Document 49
Annexure D Project Tracker 51

LIST OF FIGURES

Figure 2.1 Layered AgentOps architecture 7
Figure 2.2 Message lifecycle and failure path 8
Figure 2.3 Leader routing and delegated review 9
Figure 2.4 Prompt construction and persistent context 10
Figure 2.5 Single-host cloud deployment boundary 12
Figure 4.1 Proposed development and verification stages 17
Figure 5.1 Product context and external dependencies 21
Figure 5.2 Logical relationship of operational and project records 25
Figure 6.1 Expected direct request and delivery sequence 30

LIST OF TABLES

Table 1.1 Problem areas and required responses 2
Table 1.2 Present scope and extension boundary 4
Table 2.1 Agent concepts and responsibilities 6
Table 2.2 Background services and verification concerns 11
Table 3.1 Separation of evaluation concerns 13
Table 3.2 Proposed software workflow artifacts 15
Table 3.3 Literature synthesis and project relevance 16
Table 4.1 Work packages and review artifacts 18
Table 4.2 Project risk register 19
Table 4.3 Relative milestones and exit evidence 20
Table 5.1 User classes and principal use cases 22
Table 5.2 Execution requirements 23
Table 5.3 Collaboration and continuity requirements 24
Table 5.4 Representative API interfaces 26
Table 5.5 Nonfunctional requirements and acceptance approach 27
Table 5.6 Environment and operating constraints 28
Table 6.1 Capability summary and evidence status 29
Table 6.2 Team demonstration observation sheet 31
Table 6.3 Failure windows and expected interpretation 32
Table 6.4 Evidence required for stronger conclusions 33
Table 7.1 Test levels and evidence 34
Table 7.2 Proposed functional cases 35
Table 7.3 Proposed access and fault cases 36
Table 7.4 Evaluation measurements 37
Table 7.5 Requirement traceability and release evidence 38
Table 8.1 Future extensions and intended benefit 40

NOMENCLATURE

Term	Meaning
ACK	Acknowledgement of a delivered response
AI	Artificial intelligence
API	Application programming interface
CLI	Command-line interface
CORS	Cross-origin resource sharing
Daemon	Long-running background service
Dead letter	Work retained after automatic retry is exhausted
Harness	Tool-enabled agent runtime invoked by the daemon
JSON	JavaScript Object Notation
LLM	Large language model
PRN	Permanent registration or enrollment number
REST	Representational State Transfer
Root request	Original user request associated with delegated work
SSE	Server-sent events
VM	Virtual machine
WAL	Write-ahead logging
Workspace	Agent working directory and supporting files

CHAPTER 1 INTRODUCTION

1.1 PROJECT OVERVIEW

Cloud-Based Multi-Agent AI Orchestration Platform is a project by Team Cloud Clash, group BCCC41. AgentOps is the implementation described in the project documentation [1]. It brings multiple specialized artificial intelligence agents into a common execution environment and makes them reachable through messaging applications and a web dashboard. The platform coordinates agent activity around durable messages rather than treating each interaction as an isolated chatbot response.

A typical request may require research, code generation, review, and delivery of an artifact. These activities benefit from distinct responsibilities and a record of intermediate work. AgentOps supports agents with individual providers, models, workspaces, and instructions. Agents are grouped into teams, a leader receives team-directed requests, and delegated work is represented as additional queue entries. The same infrastructure also handles user requests and scheduled activity.

The approved project title emphasizes cloud-based orchestration. The documented implementation is presently a self-hosted, single-host daemon with a separate web client. It can be deployed on an appropriately configured cloud virtual machine, but this deployment model must not be confused with distributed execution across multiple workers. The report treats multi-node scheduling, elastic scaling, and role-based multi-user access as future extensions.

The report explains the problem, concepts, related work, project plan, requirements, architecture, documented capabilities, and verification strategy. It distinguishes implementation descriptions from measured evidence. Proposed test cases and evaluation procedures are presented without claiming that they have passed. This distinction provides a usable engineering baseline for demonstrating and further developing the project.

1.2 PROBLEM STATEMENT

Current AI applications frequently expose one conversational assistant at a time. A user may have to move information between tools, repeat project context, monitor a long task, and restart work after an interrupted session. A conversation window can show generated text without providing a reliable record of pending work or a clear distinction between an agent failure and a delivery failure.

The project addresses the need for a platform that can create specialized agents, preserve context, access external tools, and coordinate multiple roles through a common interface. The approved problem statement includes independent task execution, memory, scheduling, knowledge access, and collaboration. The platform should therefore manage not only model responses but also the operational state surrounding each response.

Three boundaries make the problem challenging. First, external channels may disconnect or redeliver messages. Second, provider processes can be slow, fail, or produce events that require normalization. Third, agents may create further work for other agents. Without correlation identifiers, bounded routing, and recoverable queue state, these boundaries make it difficult to determine what happened to a request.

The engineering problem is to connect these boundaries while keeping the platform understandable and controllable for an operator. A request should be accepted durably, assigned to the intended agent or team, executed under configured limits, and associated with a response or an inspectable failure. Recovery must preserve traceability without promising exactly-once external actions. A retry after a partially completed tool operation may repeat that operation unless additional safeguards exist.

Table 1.1 Problem areas and required responses

Problem area	**Required platform response**
Fragmented conversations	Stable agent, thread, and root request identifiers
Interrupted execution	Durable pending state and restart recovery
Manual coordination	Team routing and queued delegation
Invisible background work	Runtime status and operator controls
Repeated provider failures	Bounded retries and dead-letter inspection

1.3 MOTIVATION AND OBJECTIVES

The motivation is to make long-running agent assistance usable beyond a single interactive session. Tool-enabled language models can interact with files and external environments, but practical usefulness also depends on work persistence and visibility. ReAct demonstrates the relevance of combining reasoning with environment-directed actions [2]. This project builds the operational services needed to coordinate such agents rather than proposing a new reasoning algorithm.

The first objective is to support multiple configurable agents with clearly identified responsibilities. Each agent should have its own model configuration, instructions, and workspace. The second objective is to organize agents into teams with a leader and shared communication. A third objective is to provide access through Telegram, WhatsApp, Discord, a command-line interface, and the AgentOps Office dashboard.

Additional objectives concern continuity and control. Accepted messages should be stored before processing. Failed work should be retried within a limit and exposed when automatic recovery cannot finish it. Thread sessions and persistent Markdown memory should retain useful context. Schedules and heartbeat prompts should allow configured background activity while normal queue and execution policies remain in force.

Security objectives include authenticated access, paired channel senders, validated identifiers, protected attachment paths, and redacted configuration responses. These controls are scoped to their documented interfaces. They do not establish that provider subprocesses are isolated at the operating-system level or that an agent cannot follow malicious instructions embedded in authorized content.

The project will be assessable when a demonstrator can trace a request from ingestion to completion or failure, show correct team routing, recover accepted work after restart, and explain the state shown in the dashboard. Performance and task quality require a separate recorded evaluation. A feature being described in the implementation is not itself evidence that a latency, reliability, or accuracy target has been achieved.

1.4 SCOPE AND APPLICATION SCENARIOS

The present scope covers a single daemon, a local persistence layer, externally authenticated provider harnesses, and multiple clients. The daemon is responsible for routing, queue processing, session mapping, scheduling, and events. The browser is responsible for presentation and user actions. Messaging clients adapt channel-specific communication to the backend API. This division allows a consistent execution model across several interfaces.

One application is a development team containing a coder, reviewer, and researcher. A team-directed request goes to the leader, which can ask the researcher for context and the reviewer for feedback. Another application is recurring project maintenance, where a schedule supplies a defined prompt and a heartbeat asks an idle agent to inspect assigned tasks. Both scenarios use the same durable processing path as ordinary messages.

Research, marketing, sales, and support roles are possible configurations of the general platform described in the approved proposal. They are application opportunities rather than separately validated domain products. A support workflow, for example, would still require organization-specific data permissions, review rules, and evaluation before real customer deployment.

The scope excludes training a foundation model, proving autonomous task correctness, distributed consensus, elastic multi-node scheduling, and a full multi-tenant identity system. It also excludes guarantees that a messaging service or AI provider will remain continuously available. Availability depends on the host, credentials, network, subprocesses, and third-party services.

Table 1.2 Present scope and extension boundary

Present design	**Extension requiring further work**
One host with SQLite databases	Coordinated workers across multiple nodes
Shared local bearer-token administration	Individual users with role-based permissions
Provider command-line adapters	Stable compatibility across provider changes
Markdown memory index	Evaluated retrieval and memory versioning
Durable queue and acknowledgements	Idempotent external side effects

1.5 REPORT ORGANIZATION AND CONTRIBUTIONS

This report follows the prescribed institutional chapter order. Chapter 2 introduces the concepts and execution methods that support the implementation. Chapter 3 reviews relevant research and infrastructure foundations. Chapter 4 provides a proposed development and verification plan. Chapter 5 specifies requirements, interfaces, state, and operational constraints. Chapter 6 explains documented outputs and the limits of available result evidence. Chapter 7 defines test cases and evaluation procedures. Chapter 8 concludes with limitations and future work.

The principal engineering contribution is the composition of transport, orchestration, persistence, and provider invocation behind one backend. User messages, peer-agent delegation, schedules, and heartbeat prompts enter the same queue model. This common representation makes work easier to correlate and gives recovery mechanisms a consistent target.

A second contribution is a lightweight coordination protocol. Agents express addressed messages through bracket tags, and the orchestration layer converts those tags into queue items with lineage. A third contribution is an operator interface that combines live events, conversation history, queue controls, schedules, and project tasks. These mechanisms support inspection without requiring the operator to manage provider subprocesses directly.

The intended outcome is an explainable orchestration platform with a clear path toward verification. It is not a claim that adding agents always improves an answer. Collaboration can introduce additional latency, cost, contradictions, and message amplification. Appropriate evaluation compares artifact quality and resource use under controlled conditions.

The annexures support submission and continued project work. Annexure A contains the previously prepared research manuscript in its original five-page layout. Annexure B reserves the required location for an independently generated plagiarism report. Annexure C provides a proposed empathy chart and solution mapping. Annexure D provides a milestone and verification tracker. Neither publication status nor plagiarism clearance is inferred from the existence of these annexures.

CHAPTER 2 CONCEPTS AND METHODS

2.1 AGENTS AND SPECIALIZED ROLES

An agent in this project is a configured execution identity that combines a provider harness, a model selection, a working directory, and behavioral instructions. The harness supplies tool access such as shell commands or file operations. AgentOps supplies the surrounding scheduling, state, and communication services. The model and the orchestration platform therefore have different responsibilities.

A specialized role is defined through instructions and available resources. A reviewer may be asked to inspect a patch against stated acceptance criteria, while a researcher may collect supporting material. Merely naming an agent reviewer does not ensure independent or correct review. The quality of its work depends on context, model capability, tool permissions, and an evaluation procedure.

The system distinguishes user-to-agent communication from team-directed communication. A direct address targets one agent. A team address targets the configured leader, which can coordinate further exchanges. The leader is a routing and workflow role; it is not a separate distributed scheduler or a guarantee that all delegated tasks eventually converge.

Configuration selects execution behavior, session mapping preserves conversation continuity, and persistent memory stores project knowledge. These mechanisms should be validated separately.

Table 2.1 Agent concepts and responsibilities

Concept	**Meaning in AgentOps**
Agent	Configured provider-backed execution identity
Team	Named group with a designated leader
Workspace	Files and instructions available to an agent
Session	Provider conversation resumed for a thread
Memory	Persistent Markdown knowledge indexed for discovery
Skill	Reusable instructions and supporting resources

2.2 LAYERED CLIENT SERVER ARCHITECTURE

The platform uses a client-server model. The web dashboard, messaging integrations, and command-line interface send requests to the backend rather than editing database records directly. The backend is a long-running daemon that owns execution state and exposes a REST interface. This establishes a single place to validate requests and apply routing policy.

The implementation is organized as a TypeScript monorepo. The core package contains configuration, queue operations, invocation, memory, scheduling, and security helpers. The teams package handles addressed communication and conversation routing. The server package exposes API resources and events. The main package composes these services and runs the processor and channel supervision. Separate channel, CLI, and terminal visualizer packages serve their corresponding interfaces.

The dashboard is a separate Next.js application using React. It receives JSON state and live events, then renders conversations, team relationships, tasks, schedules, and operational controls. The documented backend and frontend use separate service ports. This separation supports independent deployment, but the operator must configure the correct API origin, credentials, and network path.

The architecture is layered and event-driven rather than a set of independently deployed microservices. Keeping this distinction explicit avoids overstating scalability. Logical package boundaries help maintainability, while all backend modules still share the availability and resource limits of the daemon host.

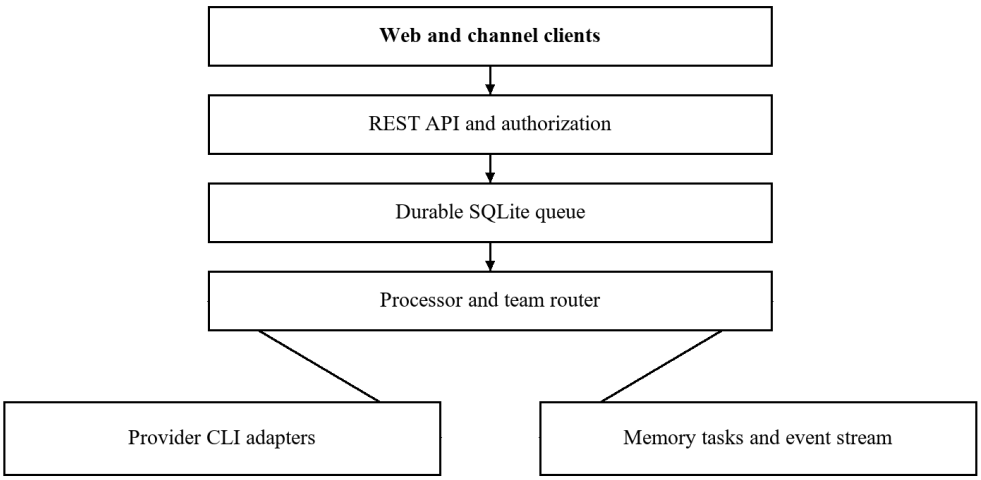


Figure 2.1 Layered AgentOps architecture

2.3 DURABLE QUEUE AND PROCESSING LIFECYCLE

The queue decouples receiving work from executing it. A client submits a request, and the backend inserts a pending message into SQLite before invoking the assigned agent. The processor discovers eligible agents and claims a message for processing. Status and retry fields make the work inspectable independently of a live subprocess.

Per-agent serialization means that one agent processes one message at a time, while distinct agents may run concurrently. This reduces simultaneous modification of one agent session. It does not protect files intentionally shared by different agents, impose a total order across a whole team, or remove the need for limits on overall concurrency.

On success, response content is stored and the input message becomes completed. On failure, retry handling records another attempt according to the implementation policy. Work that exhausts the retry limit is retained for dead-letter inspection and manual recovery. The documentation describes a retry limit of five but should be checked in the source when configuring a deployment, especially when distinguishing retries from total attempts.

SQLite persistence protects committed queue records across ordinary process restarts. It cannot guarantee the durability of uncommitted acceptance, recover a destroyed disk, or atomically roll back an external tool action. Queue state and external effects must therefore be evaluated separately.

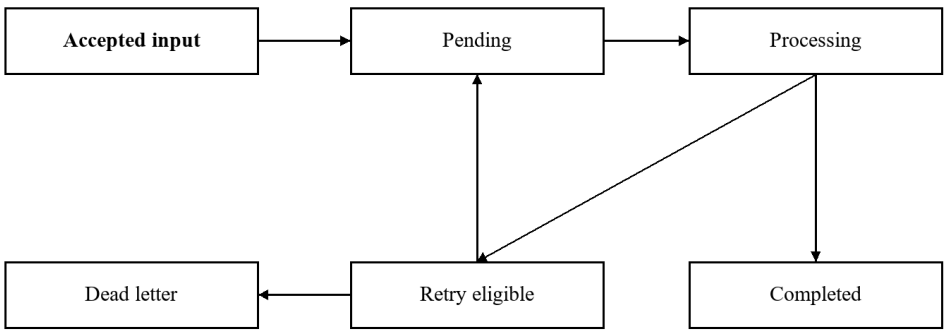


Figure 2.2 Message lifecycle and failure path

2.4 TEAM ROUTING AND COMMUNICATION

Team communication is expressed through bracket tags in agent responses. A tag such as [@reviewer: inspect the patch] requests work from one agent. A tag with multiple named recipients creates fan-out. A team-room tag such as [#dev: review ready] shares an update in the team context. Text outside tags can supply context to addressed recipients.

The parser counts bracket depth rather than ending at the first closing bracket. This matters when a delegated message contains expressions such as arr[0]. Parsing is an infrastructure concern: malformed or ambiguous tags should not silently create unintended work. Input validation and parser tests are needed to confirm behavior for incomplete tags, unknown recipients, and nested content.

Internal messages retain the originating agent, team, thread, root request, and current depth. This lineage makes it possible to group exchanges around the user's original intent. A final-notification form records communication without invoking the recipient again, which helps terminate a workflow that only requires notification.

Depth, rate, idle, and turn limits reduce runaway interaction. A finite depth alone may still allow substantial fan-out, so bounded conversations should also be evaluated by total invocations and provider usage per root request. A future per-root budget would provide a clearer cost boundary than depth by itself.

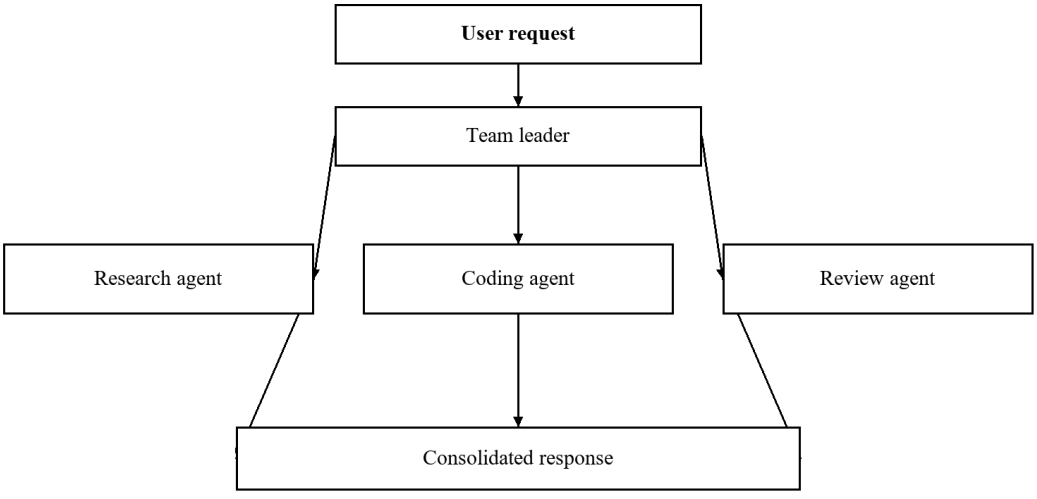


Figure 2.3 Leader routing and delegated review

2.5 MEMORY SKILLS AND CONTEXT CONTINUITY

Persistent memory is stored in Markdown files under the agent workspace. Frontmatter provides names and summaries. The memory helper constructs a bounded index for prompt inclusion, and the agent can read complete files when necessary. This makes stored knowledge discoverable without inserting every file into each prompt.

An index is a retrieval aid, not proof of accurate recall. The agent may select an irrelevant file, overlook a relevant entry, or trust an outdated statement. A useful memory policy should therefore identify an entry's source, review date, and intended scope. These provenance controls are recommended extensions unless the implementation explicitly supplies them.

Skills are reusable instruction packs with a SKILL.md entry and optional supporting resources. Workspace instructions define project behavior, and identity or soul documents describe the agent persona. These files influence execution and should be treated as configuration with a trust boundary. Content obtained from users, tools, and other agents should not automatically become persistent privileged guidance.

Provider session identifiers preserve continuity for an agent and conversation thread. They are stored with the provider identity so that one harness session is not confused with another. Long-term project memory and short-term conversation continuity should be tested independently: resuming a session does not establish that the stored project facts remain current.

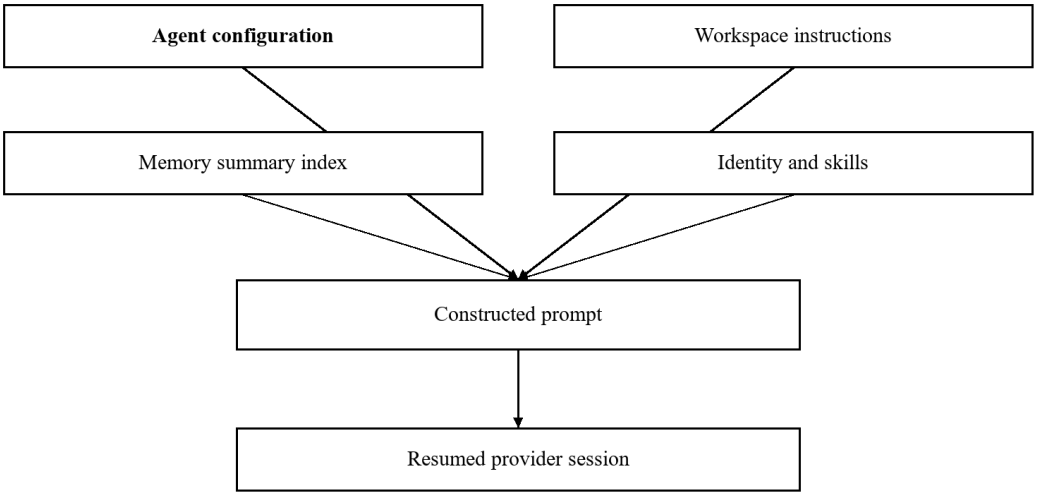


Figure 2.4 Prompt construction and persistent context

2.6 EVENTS SCHEDULING AND SUPERVISION

Internal events communicate execution changes to interested components. The dashboard receives server-sent events while ordinary user actions use REST requests. This separation fits an interface where the browser submits commands and the server streams status. Live events improve responsiveness, while database-backed API resources remain the authority for persistent state.

Schedules support recurring cron prompts and one-time execution timestamps. When a trigger fires, it creates normal queued work. A heartbeat periodically supplies configured instructions to an enabled agent and skips agents already carrying pending work. These mechanisms reuse the processor instead of creating parallel execution engines with separate state models.

The documented supervisor restarts failed channel clients. Graceful shutdown closes the databases, stops recurring services, and terminates child processes. These lifecycle behaviors should be checked under realistic failures, including hung children and delayed shutdown. Restarting a client does not prove that its last outbound message was acknowledged.

Time-based activity introduces additional operational questions. The deployment should have an explicit timezone and should record when a schedule was due, when work was enqueued, and when execution actually started. The source does not establish strict scheduling deadlines, missed-trigger replay, or fairness between interactive and scheduled requests.

Table 2.2 Background services and verification concerns

Service	**Responsibility**	**Verification concern**
Event stream	Deliver live status	Reconnect and authenticated access
Scheduler	Enqueue time-based prompts	Timezone and duplicate triggers
Heartbeat	Prompt idle agents	Busy-agent suppression
Channel supervisor	Restore failed clients	Delivery state after restart
Shutdown handler	Close resources	Orphan processes and stale state

2.7 TECHNOLOGY SELECTION AND DEPLOYMENT

TypeScript provides shared configuration and message types across backend packages. Node.js runs the daemon and launches external harnesses. Hono supplies the HTTP routing layer, and better-sqlite3 provides embedded persistence. The dashboard uses Next.js, React, and Tailwind CSS, with interaction libraries for task boards and team graphs. This stack prioritizes a small deployment footprint and direct integration with existing JavaScript tooling.

SQLite is suitable for an embedded single-host service because it avoids a separately administered database server. Write-ahead logging improves coexistence between readers and a writer, but writes remain serialized [3]. A busy timeout may reduce transient lock failures; it does not remove contention or guarantee that an overloaded service will keep up.

The documented Docker deployment uses a multi-stage build, a persistent volume, loopback port publishing, and restart behavior. The frontend is a separate service. A cloud virtual machine can host the same arrangement if networking, credentials, persistent storage, and process permissions are configured appropriately. Public access requires a deliberate secure network design rather than exposing the administrative API by default.

Operational prerequisites include installed and authenticated provider harnesses, channel credentials or sessions, sufficient disk space, and reliable time configuration. Resource sizing should come from observation of the chosen models and concurrent harnesses. The project documentation provides runtime requirements but no benchmark establishing a universal minimum RAM or CPU specification.

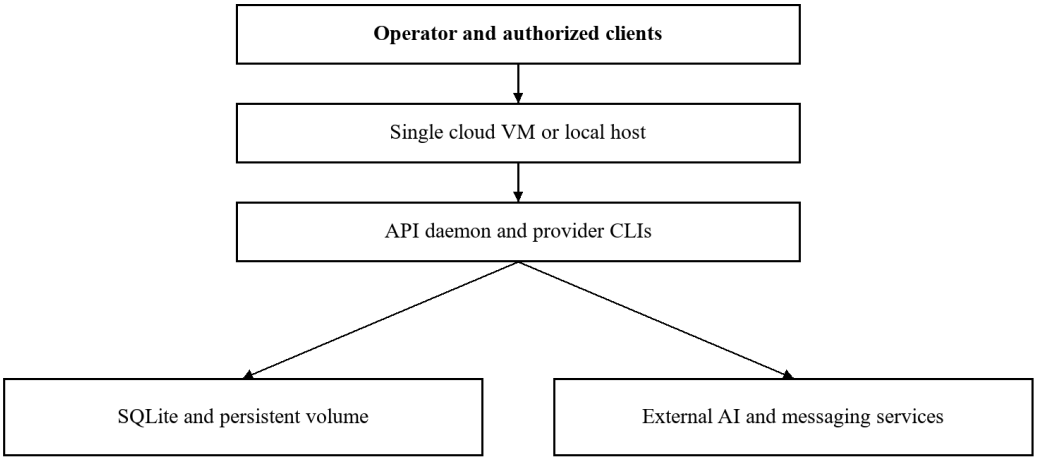


Figure 2.5 Single-host cloud deployment boundary

CHAPTER 3 LITERATURE SURVEY

3.1 TOOL ENABLED REASONING AND SYSTEM CONTEXT

ReAct studies language models that interleave reasoning with task-specific actions [2]. Its relevance is that model output can guide interaction with an environment rather than remaining a final text answer. For AgentOps, this supports the choice to orchestrate tool-enabled harnesses that can inspect files and perform project work. The orchestration layer does not reimplement the reasoning method described in that paper.

The engineering gap addressed here lies outside a single reasoning trace. A long-running service must track who requested work, which agent is responsible, what happens after a crash, and where the eventual response should be delivered. Those responsibilities remain necessary even when the underlying agent can reason effectively about an individual task.

This distinction suggests two independent evaluation questions. Infrastructure evaluation asks whether accepted messages remain traceable and whether concurrency and recovery policies hold. Application evaluation asks whether the generated artifact meets the user's acceptance criteria. A system can pass a queue recovery test while producing an incorrect patch, and a model can generate a correct patch while the channel fails to deliver it.

The proposed report methodology therefore avoids treating a language model benchmark as evidence for this platform's reliability. It uses prior research to motivate agent interaction, then defines project-specific verification for orchestration boundaries. Claims about task success, speed, and cost require controlled trials on AgentOps itself.

Table 3.1 Separation of evaluation concerns

Evaluation layer	**Example question**	**Appropriate evidence**
Reasoning and tools	Is the produced artifact correct	Acceptance checks and review
Orchestration	Was the intended agent invoked	Correlated invocation trace
Persistence	Did accepted work survive restart	Queue records before and after
Delivery	Did the user receive a response	Channel send and acknowledgement

3.2 AUTOGEN AND CONVERSATIONAL COLLABORATION

AutoGen presents a framework for applications built from agents that converse with each other [4]. Its agents can be customized and can combine language models, tools, and human participation. The framework demonstrates that conversation patterns can act as an application-building abstraction rather than merely a user interface.

AgentOps shares the use of agent conversation but places emphasis on persistent execution and external transports. A delegated message becomes a queue record, and channel delivery is tracked through outgoing response state. These are operational design choices around agent interaction. They do not establish that AgentOps provides all AutoGen capabilities or improves on its reported task performance.

For this project, the useful lesson is to make interaction roles explicit. A leader can receive a broad task, a specialist can handle a narrower request, and a human operator can inspect progress. The message schema should preserve enough context to connect these steps without requiring every participant to see all conversation content.

A fair comparison would hold provider settings, task inputs, tool permissions, and review criteria constant. The experiment should report root-level task completion, provider invocations, token usage, elapsed time, and artifact quality. Comparing the number of agents or the presence of a dashboard would not answer whether a workflow is more effective.

The design also needs explicit stopping rules. Conversational flexibility can produce repeated exchanges that do not improve an artifact. AgentOps documents depth and rate limits and a final-notification form; future evaluation should determine whether those controls produce clear outcomes on tasks requiring several review iterations.

3.3 METAGPT AND STRUCTURED ROLE WORKFLOWS

MetaGPT introduces a multi-agent framework that encodes standardized operating procedures into agent workflows [5]. It assigns different responsibilities and uses intermediate outputs to organize collaboration. Its relevance to this project is the distinction between a set of named agents and a workflow that defines what each role must produce.

AgentOps allows configurable teams, direct messages, and shared rooms. This makes the orchestration adaptable, but the documentation does not establish a complete formal procedure for every application. A software-development demonstration should therefore specify inputs, expected artifacts, review criteria, and completion conditions in addition to configuring coder and reviewer agents.

For example, a patch task can require the leader to state acceptance criteria, the coder to identify changed files, and the reviewer to report actionable defects or an explicit review outcome. The queue can preserve these exchanges, while artifact checks determine whether the intended change was actually achieved. Such a workflow is proposed here as an evaluation design rather than a recorded project result.

Structured handoffs can reduce ambiguity at the cost of additional prompts and intermediate artifacts. The appropriate degree of structure depends on the task. A short question may not benefit from several role transitions, whereas a complex implementation may need separate planning and verification. The platform should allow these choices without presenting agent count as a quality metric.

Table 3.2 Proposed software workflow artifacts

Role	**Required handoff artifact**
Leader	Task scope and acceptance criteria
Researcher	Relevant evidence with sources
Coder	Proposed change and verification notes
Reviewer	Defects or explicit review outcome
Leader at completion	Consolidated outcome and remaining limitations

3.4 INFRASTRUCTURE FOUNDATIONS AND RESEARCH GAP

SQLite write-ahead logging and standardized event streams address practical requirements surrounding collaboration. SQLite documentation explains concurrent reading with a writer and the single-writer constraint [3]. The HTML standard defines server-sent event behavior for browser clients [6]. Neither technology alone provides agent coordination, but both support the persistence and visibility needed by a service.

The combined research gap is an operational integration problem: how to connect agent teams, external messaging channels, persistent context, and recoverable work while keeping the deployment manageable. AgentOps provides one documented composition. The novelty claimed in this report is limited to that project design and its analysis, not the invention of queues, adapters, or conversational agents.

Important unresolved questions include duplicate side effects after retries, message amplification during fan-out, fairness for scheduled work, trust in persistent instructions, and authenticated browser streaming. These questions cut across module boundaries. They are therefore best evaluated with correlated traces and explicit failure injection rather than only unit tests of individual helpers.

The literature review supports a measured development approach. Existing agent frameworks motivate conversational and role-based workflows, while infrastructure references identify the limits of the chosen primitives. Project-specific experiments must then establish whether the resulting implementation behaves as intended under normal operation, restart, channel disconnects, and hostile inputs.

Table 3.3 Literature synthesis and project relevance

Reference	**Main focus**	**Relevance to AgentOps**
ReAct [2]	Reasoning with environmental actions	Tool-enabled agent execution
AutoGen [4]	Conversational agent applications	Delegation and role interaction
MetaGPT [5]	Structured collaboration procedures	Explicit handoff artifacts
SQLite WAL [3]	Embedded persistence concurrency	Queue durability and contention
HTML SSE [6]	Browser event streams	Live operational visibility

CHAPTER 4 PROJECT PLAN

4.1 DEVELOPMENT METHODOLOGY

The proposed development plan uses staged implementation with feedback at integration boundaries. Requirements and data contracts are established first, followed by the queue and provider fixture, routing, channel integration, dashboard behavior, and operational validation. This sequence makes it possible to test orchestration before depending on live model responses.

The plan is a recommended organization of work, not a reconstructed history of who implemented each feature. The project documentation identifies modules and capabilities but does not provide dated commits or verified completion records. The project tracker in Annexure D therefore records milestones and evidence requirements without inventing completion dates.

Each stage should produce an inspectable artifact. Requirements produce use cases and acceptance criteria. Design produces message schemas and state transitions. Implementation produces modules and deterministic tests. Integration produces end-to-end traces. Evaluation produces test logs, workload settings, and reviewed outputs. A stage is accepted when its evidence matches the stated exit criteria.

Feedback is expected when an integration exposes a missing contract. For example, a streaming authentication issue may require changes to both the API and browser client. Such changes should be recorded as design decisions and verified before expanding the workload. This keeps the plan responsive while preserving a clear basis for assessment.

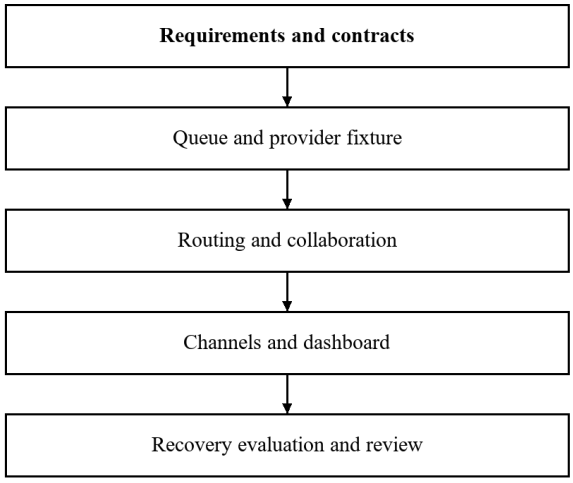


Figure 4.1 Proposed development and verification stages

4.2 WORK BREAKDOWN AND DELIVERABLES

The work breakdown separates execution infrastructure from user-facing features. This helps identify dependencies and makes review more concrete. Queue correctness must be established before fan-out and scheduling depend on it. Provider event normalization should be checked before the dashboard relies on streamed activity. Channel delivery should be verified independently from agent task quality.

The team can allocate tasks according to skills and availability, but personal contribution claims require actual records. This report identifies the three registered members without assigning unsupported implementation ownership. The guide can approve a responsibility allocation in the tracker and revise it as work progresses.

Deliverables should include both software and evidence. A runnable daemon without configuration instructions is difficult to reproduce. A dashboard screenshot without a matching trace does not establish that an action completed. A test list without logs does not establish execution. Each work package therefore names an artifact that can be checked by another person.

Table 4.1 Work packages and review artifacts

Package	**Main work**	**Review artifact**
Requirements	Actors, scope, constraints	Numbered requirements and use cases
Queue and state	Claiming, retries, recovery	State diagram and fixture traces
Provider integration	Process lifecycle and events	Adapter contract and sample streams
Team orchestration	Tags, lineage, limits	Parser and delegation cases
Channels	Ingestion, pairing, delivery	Paired sender and ACK trace
Dashboard	State views and controls	Connected UI demonstration
Operations	Deployment and recovery	Runbook and restart evidence
Evaluation	Workloads and review	Logs, metrics, and accepted artifacts

4.3 RISK MANAGEMENT AND DEPENDENCIES

The most significant dependencies are external provider harnesses and communication services. A provider can change its event format or authentication process. A channel can disconnect or throttle requests. The implementation should distinguish these failures from a queue defect so that recovery actions address the actual cause.

Operational risks include disk exhaustion, shared workspace conflicts, expired credentials, and unbounded coordination. Security risks include unauthorized senders, exposed bearer tokens, path traversal through attachments, and prompt injection. A mitigation must be evaluated at the boundary where it applies. Pairing, for example, restricts senders but does not make every message from an approved sender safe to execute.

The risk register should be reviewed when deployment conditions change. Running on a personal computer differs from hosting an administrative API on a cloud virtual machine. A larger number of concurrent agents also changes memory pressure and provider usage. No risk severity score is reported here as an observed measurement; the register is a planning tool.

Table 4.2 Project risk register

Risk	**Design response**	**Required check**
Provider format change	Adapter boundary	Replay stored event fixtures
Restart during a run	Durable queue and recovery	Inject interruption at claim boundary
Repeated external action	Stable operation identity where added	Side-effect retry experiment
Agent message amplification	Depth, rate, and time limits	Multi-recipient conversation test
Credential exposure	Redaction and restricted access	Inspect API output and logs
Channel disconnect	Supervisor and pending responses	Reconnect before and after ACK
Host or storage failure	Backups and deployment runbook	Restore configuration and databases

4.4 ACCEPTANCE MILESTONES AND DOCUMENTATION

Acceptance should proceed from deterministic behavior to live-provider demonstrations. The first milestone is a queue fixture that can accept, claim, complete, and fail work with stable identifiers. The second is correct routing and session mapping. Subsequent milestones add channels, the dashboard, scheduling, and deployment recovery. The final milestone is a reproducible demonstration against written task criteria.

No calendar commitment is implied by the relative phases below. Actual dates, owners, and review signatures belong in the team tracker. This avoids presenting a proposed plan as completed historical work. Each milestone should retain a software revision and a configuration snapshot so that a later reviewer can reproduce the observation.

Documentation should explain setup, safe credential handling, service start and stop, channel pairing, and recovery of dead-letter work. The operator should know how to distinguish an interrupted provider run from an undelivered response. A demonstration script should also state what must be reset between runs to avoid confusing reused sessions with new work.

Table 4.3 Relative milestones and exit evidence

Phase	**Milestone**	**Exit evidence**
P1	Scope and contracts agreed	Approved requirements and schema
P2	Deterministic execution path	Queue and adapter fixture logs
P3	Team communication integrated	Correlated multi-agent trace
P4	Channels and dashboard connected	End-to-end request and response
P5	Recovery and access checked	Failure injection and denial cases
P6	Submission and demonstration	Reviewed report and demo record

CHAPTER 5 SOFTWARE REQUIREMENT SPECIFICATION

5.1 PRODUCT SCOPE AND SYSTEM CONTEXT

The product provides an orchestration service for configured AI agents. An operator creates agents and teams, connects providers and messaging services, and monitors execution. Authorized users submit work through a client. External AI harnesses execute agent turns, and the backend records input, execution state, internal communication, and outgoing responses.

The service boundary includes the daemon, its databases, local configuration, and its controlled file-exchange directory. The web dashboard and channel clients interact through the API. Provider services and messaging networks remain external dependencies. A cloud host changes the location of the daemon but does not alter this logical boundary.

The current design assumes a trusted operator and a single-host storage model. The operator is responsible for selecting allowed workspaces and valid credentials. The system should reject unauthenticated API access and unpaired senders, but a role-based organization account model is outside the documented implementation.

Requirements in this chapter describe intended or documented behavior and the evidence needed to validate it. They are not declarations of test completion. Each requirement should be linked to a test case and an observable record before formal acceptance.

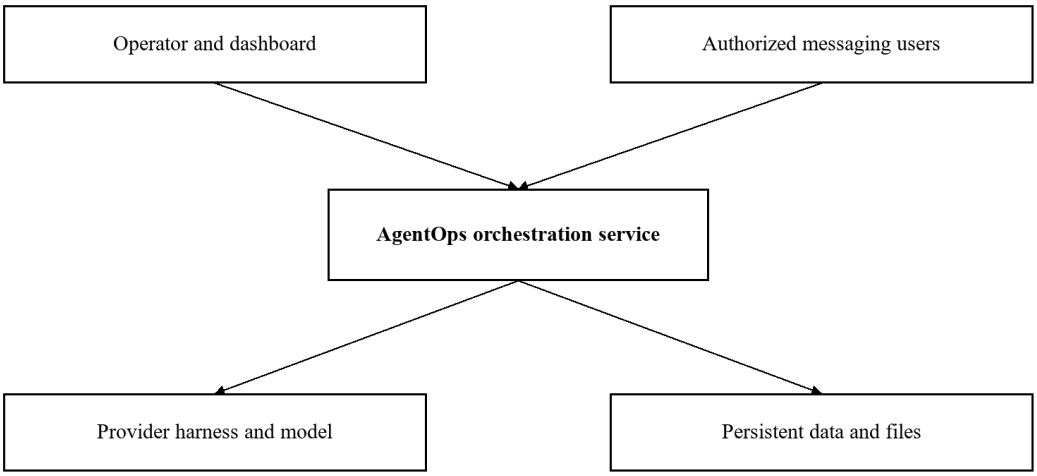


Figure 5.1 Product context and external dependencies

5.2 USER CLASSES AND USE CASES

The operator manages configuration and recovery. A paired messaging user submits work and receives responses within an approved channel. A dashboard user inspects agents, tasks, and runtime state through the configured administrative connection. A provider harness is an external execution participant rather than a human account. Scheduled triggers act as internal producers of work.

Primary use cases include sending a direct request, addressing a team leader, reviewing a conversation, creating a schedule, pairing a new sender, and retrying dead-letter work. Each use case should state its preconditions. A channel request requires a connected client and an approved sender; a provider run requires valid harness authentication and a usable workspace.

Failure paths must remain visible. If execution fails, the user interface should not imply that a successful artifact exists. If the response has not been acknowledged, the system should preserve pending delivery state. An operator action such as terminating a run should be reflected in the associated queue and runtime records.

Table 5.1 User classes and principal use cases

Actor	**Main use case**	**Expected observable outcome**
Operator	Configure an agent	Persisted provider and workspace settings
Paired user	Submit a task	Correlated message and response or failure
Team leader	Delegate a subtask	Internal message with root lineage
Dashboard user	Inspect queue state	Current processing and failed records
Scheduler	Trigger configured work	Normal queued input at trigger time
Operator	Retry dead-letter work	Traceable transition to another attempt

5.3 FUNCTIONAL REQUIREMENTS FOR EXECUTION

The execution contract starts with authenticated ingestion and ends with a persisted outcome. Routing should resolve a direct agent, a team leader, or a documented default. Before provider invocation, the processor must establish ownership of the selected message and apply per-agent serialization. Prompt construction should use the selected agent configuration and thread context.

The provider adapter should normalize relevant streamed events so the rest of the system does not depend directly on every harness event structure. Errors, interruption, and normal completion should remain distinguishable. Session identifiers should be stored against the correct agent, thread, and provider combination.

Configuration changes create a versioning question. A request can wait while an operator changes model or workspace settings. The documentation does not define a full configuration snapshot per queued request. A deployment should decide whether execution uses enqueue-time or start-time settings and should record enough information to explain a run afterward.

Table 5.2 Execution requirements

ID	**Requirement**	**Validation evidence**
FR01	Authenticate accepted API requests	Valid and invalid token cases
FR02	Persist accepted input before invocation	Commit and provider-start trace
FR03	Route direct and team requests correctly	Target agent compared with input
FR04	Allow at most one active turn per agent	Overlapping-request fixture
FR05	Invoke configured provider adapter	Captured executable and safe arguments
FR06	Preserve thread-specific sessions	Repeated thread and provider cases
FR07	Record completion or failure	Terminal queue and run records
FR08	Apply bounded retry policy	Failure sequence and dead-letter state

5.4 FUNCTIONAL REQUIREMENTS FOR COLLABORATION

Collaboration requirements extend the normal message path. A parsed delegation must identify its sender, recipient, team, thread, and original root request. A room post should be stored in the appropriate team context. A final notification should not start an unnecessary agent invocation. The parser should reject or handle malformed syntax consistently.

Memory, task management, and scheduling provide supporting capabilities. The memory index should remain within its configured size limit. Task changes should persist through the project API. A scheduled trigger should enter the same queue used for interactive input. A heartbeat should respect the documented busy-agent suppression behavior.

The interface must avoid conflating the lifecycle of a project task with that of a message. A completed message may only be an intermediate review, and a task marked done can still have a response awaiting delivery. Tests should therefore inspect the appropriate record for each requirement instead of assuming one status represents the whole workflow.

Table 5.3 Collaboration and continuity requirements

ID	Requirement	Validation evidence
FR09	Parse nested bracket payloads	Parser boundary cases
FR10	Retain root and thread lineage	Parent and child message records
FR11	Support bounded recipient fan-out	Exact expected recipient set
FR12	Record final notifications without invocation	Notification plus absent run
FR13	Build a bounded memory index	Index size and referenced files
FR14	Persist project tasks and comments	API read after update
FR15	Enqueue scheduled and heartbeat work	Trigger and queue correlation
FR16	Keep responses pending until ACK	Delivery and acknowledgement trace

5.5 DATA REQUIREMENTS AND LOGICAL MODEL

The operational data model contains messages, responses, chat messages, agent histories, and provider session mappings. The message identifier links input to an outcome. Root and thread identifiers associate a chain of delegated work with its user request. Status and retry fields support recovery, while sender and channel fields support return delivery.

Project data is held separately in projects, tasks, and comments. A task includes its project, identifier, status, assignee, and order in the board. The split between operational and project databases reduces conceptual coupling but requires explicit handling when one user action affects both stores. Cross-database atomicity is not established by the documentation.

Data retention should preserve enough history to diagnose a failure without allowing unbounded growth. The source names pruning helpers for acknowledged responses and completed messages. Retention duration, backup policy, attachment cleanup, and access to stored conversations require deployment decisions and should be included in the operator runbook.

The logical diagram is descriptive rather than a claim that every relationship is enforced by a database foreign key. Exact column constraints, indexes, and migrations should be verified against the implementation revision before schema acceptance.

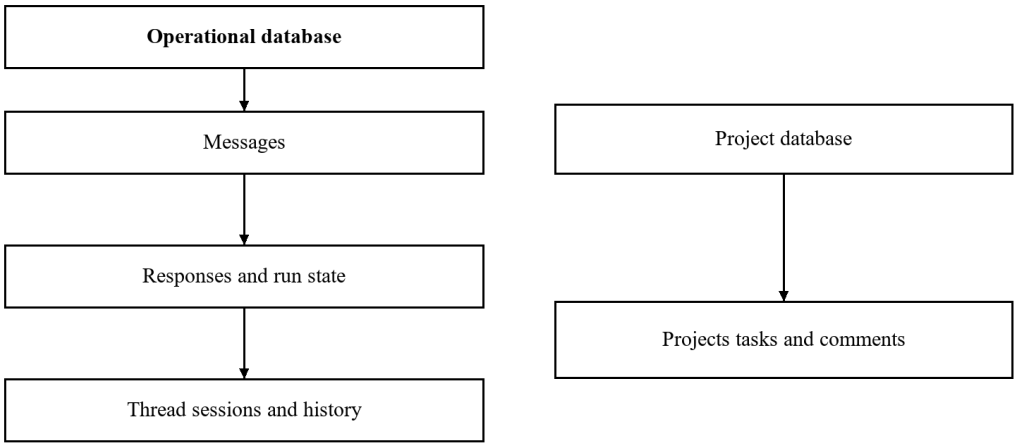


Figure 5.2 Logical relationship of operational and project records

5.6 EXTERNAL INTERFACES AND API CONTRACT

The REST API is the common interface between clients and the daemon. Configuration and execution resources should use consistent validation and error responses. The documented endpoints cover messages, agents, teams, queue inspection, responses, projects, tasks, schedules, settings, pairing, and streamed events.

Client behavior must account for failure after submission. A timeout at the client does not establish that the backend rejected the message. Repeating the request without a stable identity can create duplicate accepted work. The documentation does not establish universal request idempotency, so this should be an explicit integration concern rather than an assumed API property.

SSE provides live browser updates [6]. Native EventSource cannot attach arbitrary authorization headers. The selected authentication path for the event stream must therefore be checked against the actual frontend implementation. REST authentication success alone does not prove that the browser can open an authenticated stream.

Table 5.4 Representative API interfaces

Area	**Representative operation**	**Purpose**
Messages	POST /api/message	Accept work for an agent or team
Events	GET /api/events/stream	Stream runtime updates
Queue	GET /api/queue/status	Inspect current queue state
Dead letters	GET /api/queue/dead	Inspect exhausted work
Responses	GET /api/responses/pending	Obtain undelivered replies
Delivery	POST /api/responses/:id/ack	Acknowledge response delivery
Tasks	POST /api/tasks	Create project task
Schedules	POST /api/schedules	Register time-based work
Pairing	POST /api/pairing/approve	Approve a new channel sender

5.7 NONFUNCTIONAL AND SECURITY REQUIREMENTS

Reliability requirements concern committed input, recoverable ownership, bounded retry, and clear terminal states. Performance requirements should be expressed as measured service targets for a declared workload, not unsupported response-time promises. Maintainability depends on typed module boundaries and provider fixtures that make integration changes observable.

Access controls include a bearer token, origin restrictions, sender pairing, identifier validation, secret redaction, and allowed-root attachment checks. Each control should have a denial test. Realpath containment for attachments does not prove operating-system isolation for the entire agent subprocess. Deployment permissions, mounted files, and inherited environment variables remain relevant.

Privacy also extends beyond local storage. A configured provider may receive prompts, memory excerpts, and file content. Self-hosting the daemon does not imply that inference remains on the host. The operator should select data and providers according to the intended use case and should restrict persistent logs to necessary information.

Table 5.5 Nonfunctional requirements and acceptance approach

ID	**Requirement**	**Acceptance approach**
NF01	Traceability of accepted work	Reconcile all accepted identifiers
NF02	Recovery after daemon restart	Controlled interruption experiment
NF03	Bounded agent activity	Depth, rate, and timeout checks
NF04	Authenticated administration	Unauthorized access denied
NF05	Attachment path containment	Traversal and symlink test cases
NF06	Configuration secret redaction	Inspect responses and error logs
NF07	Operational observability	Correlate queue, runs, and delivery
NF08	Reproducible deployment	Clean setup using recorded versions

5.8 HARDWARE SOFTWARE AND OPERATING CONSTRAINTS

The documented backend uses Node.js version 22.12 or later and TypeScript. The environment must support building the native SQLite binding when a prebuilt package is unavailable. Provider harnesses must be installed and authenticated independently. Telegram and Discord require their respective bot credentials, while the documented WhatsApp client uses a browser-backed session.

No fixed hardware requirement is established by the source. A practical sizing exercise should measure memory and CPU use with the selected number of concurrent harnesses, database activity, and dashboard load. Storage planning should include queue records, conversations, run data, exchanged files, and backups. A cloud deployment should preserve data across process or container replacement.

The service should have a declared timezone, allowed origins, workspace roots, and credential sources. The documented default ports are backend 3777 and frontend 3000. These are deployment defaults rather than reasons to expose either administrative service publicly. Network access should be chosen to match the intended users and trust boundary.

Table 5.6 Environment and operating constraints

Component	**Documented requirement or constraint**
Backend runtime	Node.js 22.12 or later
Build tooling	TypeScript and npm workspace build
Database	SQLite through better-sqlite3
Provider	Installed and authenticated compatible CLI
Browser frontend	Separate Next.js application
Container build	Native compilation tools where required
Persistence	Writable durable data directory or volume
Availability	Depends on host, network, channels, and provider

CHAPTER 6 RESULTS

6.1 DOCUMENTED IMPLEMENTATION OUTCOMES

The project documentation describes a functioning architecture with multi-agent configuration, team routing, messaging clients, a web dashboard, persistent storage, and operational controls. These are the implementation outcomes available for analysis. The supplied material does not include an independently executed repository build, dated test logs, deployment traces, or measured performance results.

Accordingly, this chapter reports documented capabilities and their expected observable outputs. It does not convert the existence of a module into a passed acceptance test. A demonstration should connect the listed capability to a recorded action and resulting state. That evidence can then replace the verification status in the project tracker.

The most important structural outcome is the common execution path. User requests, internal delegation, schedules, and heartbeats all produce work for the queue. This makes the root request and execution history central to explaining a result. A user-visible answer should be assessed alongside its delegated runs and delivery state.

Table 6.1 Capability summary and evidence status

Capability	**Expected output**	**Evidence in available material**
Agent configuration	Named provider-backed agents	Described in project documentation
Team delegation	Child messages with lineage	Protocol and schema described
Durable processing	Queue state across lifecycle	Storage and recovery described
Messaging access	Requests and delivered replies	Client integrations described
Dashboard control	Live state and operator actions	Routes and components described
Measured performance	Recorded latency and throughput	Not supplied
Executed test results	Logs tied to a revision	Not supplied

6.2 EXPECTED DIRECT REQUEST DEMONSTRATION

A direct-request demonstration should begin with an authenticated client and a configured agent. The user submits a small task with an unambiguous acceptance condition, such as producing a short explanation or generating a bounded file artifact. The demonstrator records the accepted message identifier and the selected thread before observing provider execution.

The expected sequence is durable insertion, claim, provider start, streamed activity, response storage, channel delivery, and acknowledgement. Each observation should be tied to the same request. If execution fails, the demonstrator should inspect the retry or dead-letter state instead of treating an error message as successful task completion.

A file-producing task adds an attachment boundary. The agent may request file delivery, but the backend should validate the resolved file path against allowed roots. The demonstration should show both an allowed file and a rejected out-of-root request. This establishes the scope of the attachment interface without implying that every tool operation is sandboxed.

The expected sequence below is a logical demonstration flow, not a captured runtime trace. Actual screenshots and timings should be taken from the running project, with credentials and unrelated personal information removed. Invented interface screenshots would not provide implementation evidence.

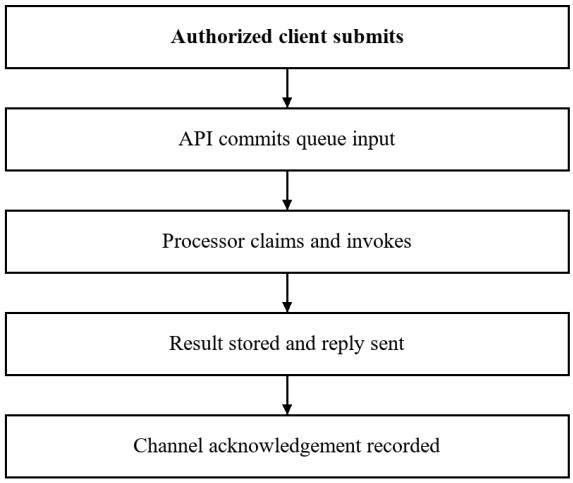


Figure 6.1 Expected direct request and delivery sequence

6.3 EXPECTED TEAM COLLABORATION DEMONSTRATION

A team demonstration should use a task that benefits from a separate review step. The leader receives the user's request and states acceptance criteria. It delegates a bounded subtask, receives a result, and requests review where necessary. The final outcome should identify the artifact and any unresolved issue rather than merely showing that several agents exchanged messages.

The platform should preserve the root identifier across the delegated messages. It should also show the sender and recipient of each handoff. These fields allow the reviewer to distinguish a child invocation from an unrelated task. Per-agent serialization should hold even if more than one team targets the same agent.

The demonstration should include a final notification that does not cause another invocation. It should also include a controlled depth or rate-limit boundary. This tests termination and containment, which are separate from whether the agents can produce useful content.

Collaboration is not assumed to improve quality. A baseline run with one agent should use the same task, provider settings, and acceptance criteria. Compare reviewed artifact quality, elapsed time, and total provider invocations. This shows whether the added review step is justified for that workload.

Table 6.2 Team demonstration observation sheet

Observation	**Record to retain**
User request accepted	Root message and thread identifiers
Leader selected	Team configuration and target agent
Subtask delegated	Child message with sender and recipient
Review completed	Review artifact and associated run
Limits applied	Depth, rate, or timeout boundary outcome
Final response delivered	Response identifier and acknowledgement

6.4 RELIABILITY AND RECOVERY OBSERVATIONS

The architecture suggests that committed pending messages can survive an ordinary daemon restart. Startup recovery resets stale processing work and closes abandoned run records. Outgoing responses remain pending until acknowledged. These are meaningful recovery mechanisms, but their exact behavior should be observed at controlled interruption points.

Two failure windows are especially important. If an agent performs an external action and fails before queue completion, retrying can repeat that action. If a client delivers a response and fails before acknowledgement, the response can be delivered again. The design therefore supports at-least-once behavior in relevant paths rather than establishing exactly-once real-world effects.

Recovery results should state what failed, when it failed, which records existed before restart, and what changed afterward. A successful process restart is not enough: the evaluator must reconcile accepted message identifiers and inspect duplicate effects. The absence of a lost message in a small demonstration also does not establish a universal availability guarantee.

Table 6.3 Failure windows and expected interpretation

Interruption point	**Expected interpretation**
Before queue commit	Durable acceptance is not established
After commit before claim	Pending work should remain discoverable
During provider execution	Stale ownership and partial effects require review
After side effect before completion	A retry may duplicate the effect
After send before acknowledgement	A reply may be delivered again
After acknowledgement	Delivery should remain recorded subject to retention

6.5 DISCUSSION OF RESULTS AND LIMITATIONS

The documented design brings execution control and user access into one operational model. Its queue and lineage fields provide a basis for explaining what happened to a request. The dashboard and API offer several views of that state. The value of this composition depends on those views agreeing under restart, delayed events, and channel reconnects.

Several limitations remain material. SQLite and a single daemon concentrate write coordination and host availability. Provider harnesses are external dependencies whose behavior can change. The unofficial WhatsApp integration adds session-management risk. Shared bearer-token administration does not provide the same separation as individual users and roles. Persistent memory can retain incorrect instructions as well as useful knowledge.

There are also limits to what the available evidence can support. No numerical latency, accuracy, cost saving, uptime percentage, or throughput improvement is reported. The evaluation plan in Chapter 7 defines how such evidence should be collected. Until those experiments are executed, the strongest supported conclusion is that the project provides a documented integration architecture with testable operational properties.

A submission demonstration should retain the project revision, configuration, accepted request list, queue snapshots, provider traces, and redacted delivery records. Those artifacts make the result reviewable and help identify whether a failure belongs to routing, execution, persistence, application quality, or transport.

Table 6.4 Evidence required for stronger conclusions

Proposed claim	**Required evidence**
Lower response time	Controlled baseline and latency distribution
Better task quality	Fixed acceptance criteria and independent review
Improved recovery	Repeated failure injection and reconciliation
Lower operating cost	Provider usage and comparable workload
Safe attachment access	Traversal, symlink, and authorization cases

CHAPTER 7 SOFTWARE TESTING

7.1 TEST STRATEGY AND ENVIRONMENT

Testing should separate deterministic orchestration behavior from stochastic model output. The project documentation names backend tests using Node's built-in test runner and a fake provider fixture, and frontend tests using tsx with jsdom. It also names TypeScript checking and ESLint. These tools form a test strategy; no execution status is inferred from their presence.

Fixture tests should emit controlled normal events, malformed events, delays, failures, and interruption behavior. They can validate routing, queue transitions, session mapping, and limits without provider credentials or model cost. Live-provider tests should then verify actual harness compatibility and end-to-end task outcomes under recorded configuration.

The environment record should include the source revision, operating system, runtime versions, provider version and model, host resources, database settings, enabled channels, timezone, and sanitized configuration. Tests that reuse sessions should state that fact. Otherwise, prior context may change an outcome and make the result difficult to reproduce.

Table 7.1 Test levels and evidence

Level	**Main target**	**Evidence to retain**
Static checks	Types and lint rules	Command output and revision
Unit tests	Parsing and helper behavior	Named cases and assertions
Component tests	Queue and adapter lifecycle	Fixture trace and database state
Integration tests	API, channels, and dashboard	Correlated request and response
Fault injection	Recovery and duplicate windows	Interruption point and reconciliation
Live task evaluation	Artifact quality and cost	Acceptance results and usage

7.2 FUNCTIONAL TEST CASES

Functional cases should have explicit inputs, preconditions, expected outcomes, and recorded actual outcomes. The table lists proposed cases for the main execution path. Their status remains unexecuted until logs tied to a source revision are retained. A test should not be marked passed merely because the corresponding requirement is described in the report.

Routing cases should include direct agents, team leaders, defaults, and unknown recipients. Parser cases should cover nested brackets and multiple recipients. Concurrency cases should submit overlapping work for the same agent and independent work for different agents. Session cases should vary both the thread and provider identity.

The assertion should inspect the authoritative state. A visual typing indicator is not sufficient for per-agent serialization. A response text is not sufficient for an acknowledgement check. This approach reduces tests that pass because the interface looks plausible while the stored state is incorrect.

Table 7.2 Proposed functional cases

Case	**Input or action**	**Expected outcome**
FT01	Direct agent address	Intended agent is selected
FT02	Team address	Configured leader is selected
FT03	Two requests to one agent	No overlapping active turns
FT04	Independent agent requests	Concurrency where resources allow
FT05	Nested brackets in delegation	Full intended payload is retained
FT06	Multiple recipients	Correct child set and lineage
FT07	Final notification	Stored message without extra invocation
FT08	Same thread and provider	Correct session is resumed
FT09	Different thread or provider	Session context is not mixed
FT10	Send followed by ACK	Pending response becomes acknowledged

7.3 SECURITY AND FAILURE TEST CASES

Security cases should test denied access as carefully as successful access. Invalid tokens, unpaired senders, malformed identifiers, and disallowed attachment paths should be rejected before unauthorized provider work occurs. The evaluator should inspect both the HTTP or channel response and the absence of an unintended invocation.

Fault injection should target specific lifecycle boundaries. Stop the daemon after queue insertion, terminate a provider during execution, and interrupt delivery before acknowledgement. Also test a slow but healthy provider so that stale-recovery policy does not mistakenly create overlapping ownership. Each case should reconcile all accepted identifiers after restart.

Prompt injection requires separate adversarial cases. A message or retrieved file can contain instructions attempting to override task scope or expose data. Authentication does not remove that risk. The test should record available tools and permissions and inspect whether untrusted content changes privileged behavior.

Table 7.3 Proposed access and fault cases

Case	**Condition**	**Required observation**
SF01	Invalid bearer token	Denial and no provider invocation
SF02	Unpaired channel sender	Pairing flow or denial
SF03	Attachment traversal path	Access outside allowed roots denied
SF04	Attachment through symlink	Canonical containment enforced
SF05	Repeated provider failure	Retry limit and dead-letter behavior
SF06	Restart after committed input	Accepted record remains traceable
SF07	Crash after external action	Possible duplicate effect documented
SF08	Crash after channel send	Pending ACK and redelivery inspected
SF09	Slow healthy provider	Ownership remains consistent
SF10	Hostile retrieved instructions	Tool boundary and data exposure reviewed

7.4 PERFORMANCE AND QUALITY EVALUATION

Performance evaluation should record timestamps at enqueue, claim, provider start, provider end, and delivery. These boundaries distinguish queue waiting from model execution and transport delay. Report median and 95th percentile latency rather than a single convenient example. Throughput should count completed root requests, not all child messages created by fan-out.

Vary agent count and accepted arrival rate while holding task inputs and provider settings constant. Include both evenly distributed requests and a workload concentrated on one agent. Per-agent serialization can produce a hotspot even when other agents are idle. Queue length over time reveals whether the tested rate is sustainable.

Quality evaluation requires task acceptance criteria. For a code task, retain the changed artifact, relevant checks, and independent review. For a research task, check source relevance and factual support. Compare a single-agent baseline with a team workflow using the same cost limit. More messages or more agents should not be scored as a better answer.

Table 7.4 Evaluation measurements

Measurement	**Interpretation**
Queue waiting time	Delay between acceptance and claim
End-to-end latency	Delay between acceptance and delivery
Root throughput	Completed user requests per interval
Invocations per root	Coordination amplification
Retry and dead-letter counts	Execution failure burden
Duplicate delivery count	Transport replay behavior
Token usage and provider cost	Resource demand for comparable work
Accepted artifact count	Task success under fixed criteria

7.5 TRACEABILITY AND RELEASE ACCEPTANCE

Release acceptance should connect requirements to tests and evidence. A requirement is accepted when its observable criterion is satisfied on a recorded revision, not when its implementation is merely present. Failed and blocked cases should retain their input, error, and follow-up action so that another team member can reproduce the issue.

The release record should identify configuration assumptions, known limitations, and unresolved defects. It should also preserve the demonstration workload and data-reset procedure. A dashboard view may contain stale client state, so any inconsistency should be checked against persisted API resources before drawing a conclusion.

The traceability matrix below is an initial mapping. It can be extended with actual test identifiers and evidence links as the project is exercised. The report intentionally leaves actual outcomes to the test record rather than inventing a pass percentage.

Table 7.5 Requirement traceability and release evidence

Requirement group	**Proposed test mapping**	**Required acceptance evidence**
Authenticated ingestion FR01	SF01 and SF02	Denial and authorized request traces
Routing FR03	FT01 and FT02	Expected target agent records
Serialization FR04	FT03 and FT04	Non-overlapping per-agent runs
Session mapping FR06	FT08 and FT09	Correct session association
Team protocol FR09 to FR12	FT05 to FT07	Parser and lineage evidence
Response ACK FR16	FT10 and SF08	Pending-to-ACK transition
Recovery NF02	SF05 to SF09	Reconciled work after interruption
Path containment NF05	SF03 and SF04	Denial without file disclosure

CHAPTER 8 CONCLUSION AND FUTURE WORK

8.1 CONCLUSION

Cloud-Based Multi-Agent AI Orchestration Platform addresses the operational coordination needed around tool-enabled AI agents. The AgentOps implementation combines provider-backed agents, team communication, durable queueing, persistent context, schedules, messaging clients, and a separate web dashboard. It gives accepted work an identity and a lifecycle that can be inspected independently of one conversation window.

The design is strongest where it applies one execution model to several producers of work. User input, delegated messages, schedules, and heartbeat prompts enter the same queue. Provider adapters separate harness-specific behavior from the processor. Channel acknowledgements distinguish response creation from recorded delivery. These boundaries give the project a clear basis for testing and further improvement.

The report also identifies the limits of that basis. Persistence does not guarantee exactly-once external actions. Team communication can amplify cost and latency. Authentication does not remove prompt injection. Local hosting does not ensure local inference, and a cloud virtual machine does not by itself create distributed orchestration. These distinctions are necessary for an accurate explanation of what the current project supports.

The present evidence supports a documented architecture and an explicit verification plan. It does not support numerical performance or quality claims. A complete demonstration should execute the functional and failure cases, retain correlated traces, and evaluate the produced artifacts against written criteria. That work will establish which intended properties hold in the actual deployment.

The resulting project provides a concrete integration platform for studying durable and observable multi-agent assistance. Its next engineering steps are well defined: verify the current path, record operational evidence, and extend the design only where measured needs and application requirements justify the change.

8.2 FUTURE WORK

The first priority is stronger control over repeated actions. Stable operation identifiers and idempotent external integrations would reduce duplicate effects after retries. A per-root execution budget would complement depth and rate limits by constraining the total amount of delegated work. Structured completion criteria would help the operator determine when a conversation has achieved its task.

A second priority is access separation and execution isolation. Role-based user permissions would make the platform more suitable for multiple independent users. Subprocess permissions, workspace mounts, and credential scope should be evaluated explicitly. An authenticated SSE path should be covered by an integration test so that browser monitoring works under the deployed access model.

Memory improvements should focus on correctness as well as retrieval convenience. Versioned entries, provenance, review dates, and measured retrieval quality would help prevent persistent stale guidance. Provider cost reporting and per-root usage summaries would make team workflows easier to compare with single-agent baselines.

Distributed execution is a longer-term extension. It would require coordinated worker ownership, a shared persistence strategy, recovery rules for network partitions, and migration of operational state. Replacing SQLite with another database alone would not solve these problems. The design should evolve from observed bottlenecks and validated correctness requirements.

Table 8.1 Future extensions and intended benefit

Extension	**Intended benefit**
Idempotent operation tracking	Fewer duplicate external effects
Root request budgets	Predictable coordination cost
User roles and scoped credentials	Separation of administrative authority
Isolated execution environments	Clearer tool and filesystem boundaries
Versioned and evaluated memory	More reliable project context
Usage and cost dashboards	Evidence-based workflow selection
Coordinated multi-node workers	Capacity beyond one daemon host

BIBLIOGRAPHY

[1] AgentOps, “Complete Project Documentation,” unpublished project documentation supplied as PROJECT_DOCUMENTATION.md, n.d.

[2] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” arXiv:2210.03629, 2022, doi: 10.48550/arXiv.2210.03629.

[3] SQLite, “Write-ahead logging.” Accessed: Oct. 17, 2026. [Online]. Available: https://sqlite.org/wal.html

[4] Q. Wu et al., “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” arXiv:2308.08155, 2023, doi: 10.48550/arXiv.2308.08155.

[5] S. Hong et al., “MetaGPT: Meta programming for a multi-agent collaborative framework,” arXiv:2308.00352, 2023, doi: 10.48550/arXiv.2308.00352.

[6] WHATWG, “Server-sent events,” HTML Living Standard. Accessed: Oct. 17, 2026. [Online]. Available: https://html.spec.whatwg.org/multipage/server-sent-events.html

ANNEXURE A

LIST OF PUBLICATIONS AND RESEARCH PAPER

A research manuscript associated with the project is included on the following five pages. Its original two-column layout is retained. The manuscript presents AgentOps as a system-design study and describes an evaluation protocol without claiming measured experimental results.

Item	**Manuscript record**
Title	AgentOps A Self-Hosted Platform for Durable Multi-Agent Collaboration Across Messaging Channels
Authors	Tanishq Sachin Sonawane; Varun Hitendra Patil; Keyur Arvind Mhatre
Format	IEEE-style two-column manuscript of five pages
Status	Unpublished manuscript prepared for project review
Publication details	No journal or conference acceptance is recorded

The manuscript uses its own reference numbering. Annexure inclusion records the availability of the paper; it does not establish publication or peer review.

AgentOps: A Self-Hosted Platform for Durable Multi-Agent Collaboration Across Messaging Channels

Tanishq Sachin Sonawane Varun Hitendra Patil Keyur Arvind Mhatre

Abstract—Large language model agents can perform tool-assisted tasks, but sustained collaboration also requires durable execution, communication routing, persistent context, and operational visibility. This paper presents AgentOps, a self-hosted platform that connects teams of specialized agents to Telegram, WhatsApp, Discord, a command-line interface, and a web dashboard. Its architecture combines a SQLite-backed queue, per-agent serialized execution, provider-specific command-line adapters, and event-driven status updates. A lightweight coordination protocol converts agent-addressed messages into queued work while preserving conversation lineage. Persistent memory, scheduled tasks, and heartbeat prompts support activity beyond an individual chat session. This paper develops the system architecture, analyzes delivery and coordination properties, and specifies a reproducible evaluation protocol. The evidence is limited to supplied project documentation; no performance measurements or independently executed tests are claimed. The analysis identifies duplicate side effects, message amplification, and single-host capacity as important constraints. AgentOps provides a concrete engineering design for studying durable, observable multi-agent assistance in a self-hosted setting.

Index Terms—Multi-agent systems, large language models, durable queues, agent orchestration, self-hosting, messaging platforms.

I. INTRODUCTION

Tool-enabled language model agents extend conversational assistance into activities such as coding, research, and file manipulation. ReAct illustrates the value of interleaving reasoning with actions that access an environment [1]. However, an agent capable of completing one interaction is only one component of an assistant intended to operate over long periods. Such a system must retain requests, manage failures, preserve context, and expose progress to users.

A team-based workflow adds another requirement: agents with distinct responsibilities must exchange work without losing its relationship to the original request. A coding agent may request review, a reviewer may return corrections, and a leader may summarize the outcome. These exchanges compete with user messages, scheduled work, and channel delivery. Treating every exchange as an immediate process invocation makes recovery and resource control difficult.

AgentOps addresses these concerns by placing a durable work queue between message ingestion and agent execution. It integrates collaborative agents with everyday messaging channels and a browser-based control interface. The focus is the operational architecture around existing agent harnesses rather than a new language model or training method.

The central research question is: how can a single-host assistant coordinate heterogeneous agents across multiple channels while retaining recoverable work state and inspectable execution? The design pursues three contributions. First, it separates channel transport, durable execution, and provider invocation through explicit interfaces. Second, it represents team communication as queued messages carrying conversation lineage. Third, it combines coordination limits, persistent context, and operational controls in one deployment model.

This is a system-design paper grounded in project documentation [2]. Component behavior is reported as documented, and reliability implications are derived analytically. The source describes a test structure but provides no test reports, workload traces, or benchmark dataset. Consequently, the paper presents evaluation procedures and expected invariants instead of unsupported numerical results.

II. BACKGROUND AND RELATED WORK

AutoGen supports applications in which customizable agents converse and combine language models, tools, and human participation [3]. It establishes multi-agent conversation as a flexible application-building abstraction. AgentOps adopts conversational delegation but emphasizes how messages enter through external channels, survive process failures, and remain visible to an operator. This is a difference in engineering emphasis, not evidence of superior task-solving performance.

MetaGPT organizes agent roles using standardized operating procedures and structured workflows [4]. This motivates explicit responsibilities and intermediate review. AgentOps instead documents configurable teams with a leader, direct messages, and shared rooms. Such flexibility can support varied workflows, but it does not by itself ensure artifact quality or enforce a complete software-development procedure.

At the infrastructure level, SQLite write-ahead logging permits readers to operate alongside a writer, while writes remain serialized [5]. This suits an embedded single-host service but does not imply unrestricted write concurrency. Server-sent events (SSE) provide a standardized server-to-browser event stream [6]. AgentOps combines SSE for live status with ordinary HTTP requests for user actions.

AgentOps therefore integrates established techniques rather than introducing a new model. Its contribution is the operational composition and its tradeoffs. Relative reliability, speed, or cost cannot be established without equivalent workloads and deployment conditions.

III. SYSTEM ARCHITECTURE

A. Requirements and Component Boundaries

The platform targets a self-hosted installation in which one operator configures agents, teams, providers, and communication channels. Functional requirements include receiving a request from multiple clients, routing it to an agent or team, invoking an external agent harness, coordinating follow-up work, and delivering a response. Operational requirements include durable pending work, failure visibility, bounded execution, and recoverable conversation state.

The backend is a long-running TypeScript and Node.js daemon. Its modular packages separate core infrastructure, team orchestration, HTTP services, channel clients, and application startup. A Hono-based REST interface exposes configuration and runtime operations. The standalone Next.js and React dashboard communicates with the backend using JSON requests and SSE; it does not directly access operational databases.

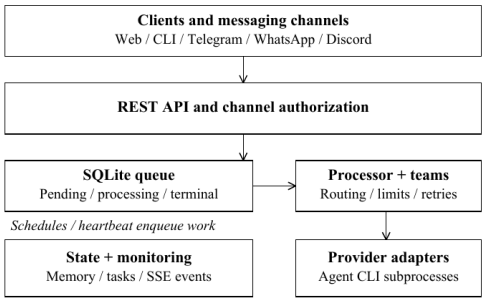


Fig. 1. Logical AgentOps architecture. All execution and internal delegation pass through the durable work queue; SSE carries status to the dashboard.

Figure 1 emphasizes the persistence boundary. A received request becomes durable work before an agent is invoked. Channel clients and the web interface share the same backend contract, allowing transport-specific behavior to remain outside orchestration. The database and filesystem remain backend responsibilities, simplifying the ownership of state.

B. Storage and Conversation Context

The records serve different purposes: queue status tracks execution intent, while chat history records conversation content. A visible reply is not sufficient evidence that every delegated action completed. Interfaces should correlate both views using stable identifiers when explaining the outcome of a root request.

The operational database, agentops.db, stores incoming messages, outgoing responses, team-room history, per-agent chat history, and provider session identifiers. Message records include an identifier, target agent, channel, status, retry counter, and lineage metadata. The session mapping is keyed by agent, conversation thread, and provider, allowing a harness session to resume within the appropriate context.

A second SQLite database, tasks.db, stores projects, tasks, and comments for the kanban interface. Task states span backlog, todo, in-progress work, review, and completion. Separating task records from the execution queue distinguishes project tracking from message processing. The documentation does not establish a transaction spanning the two databases, so atomic consistency between a task update and a queue transition must not be assumed.

Configuration resides in a local settings file. Each agent also has a working directory containing instructions, identity information, memory documents, and optional skills. These file-based resources make behavior editable without database schema changes. Their validity and trustworthiness remain separate concerns from the durability of queue records.

C. Provider Adapters and User Interfaces

Provider adapters construct command-line arguments and normalize streamed events from Claude Code, Codex, or OpenCode. The daemon launches the selected harness as a child process in the agent workspace. Thus, the platform orchestrates existing tool-enabled runtimes rather than implementing all tools itself. Custom endpoints reuse a compatible harness configuration, so compatibility still depends on that harness and its event format.

The dashboard supplies conversation views, a team graph, a kanban board, schedules, and runtime controls. SSE forwards execution activity to the browser [6], while HTTP routes handle commands and state changes. The documented browser implementation requires attention to authentication: native EventSource does not accept arbitrary authorization headers. A deployment must verify a compatible authenticated streaming path instead of assuming ordinary REST header handling automatically applies to SSE.

D. Deployment Model

State reconciliation is especially relevant after a browser reconnects. An event stream conveys changes observed while connected, whereas persisted REST resources describe current state. The dashboard should refresh authoritative records after reconnection and avoid treating a transient animation or typing indicator as a durable execution outcome. Replay guarantees and event retention are not specified in the source and require separate validation.

The documented deployment uses a multi-stage Docker build, a persistent data volume, process restart behavior, and a backend port published on the loopback interface. Messaging integrations run as supervised clients. The web frontend is a separate application, and AI harnesses must be installed and authenticated. Local persistence does not make model inference local: prompts and context may still be transmitted to the configured provider.

This boundary favors small-team and personal deployments where operating simplicity matters. It also concentrates availability in one host, one daemon, and the associated storage. A container restart can restore a process, but host failure, exhausted disk capacity, and inaccessible provider credentials require additional operational measures.

IV. MESSAGE PROCESSING AND COORDINATION

A. End-to-End Execution

An authorized client submits a message to the backend. The request is stored with pending status and triggers a processor wake-up. The router resolves an explicit agent address or a team address; team requests go to the configured leader. The processor claims work and marks it as processing. Incoming hooks may transform the message before prompt construction and provider invocation.

Prompt construction combines agent instructions, teammate information, a memory index, and identity material. The selected adapter starts a harness session or resumes the session associated with the thread. Streaming events become common runtime events for monitoring. On completion, the team module extracts internal communication, outgoing response content is persisted, and the input reaches a terminal state. Provider failures trigger retry handling and ultimately dead-letter handling when the configured limit is exhausted.

For a request traversing the system, total response time can be decomposed into queue waiting, prompt preparation, provider execution, coordination, and channel delivery. A slow reply therefore need not indicate slow model inference. Timestamping these boundaries would let an operator distinguish overloaded scheduling from provider latency or a disconnected messaging client. Internal delegation adds further queue and execution intervals, so reporting only one provider call would understate the user-visible cost of a team workflow.

For agent a, the documented scheduling policy imposes the following local concurrency constraint:

$$active(a, t) \leq 1. \quad (1)$$

Different agents may execute concurrently, so global throughput can improve when work is distributed across independent agents. Equation (1) reduces competing modifications within one agent session. It does not guarantee a total ordering across agents, thread isolation inside shared files, or freedom from contention at the provider and database layers.

B. Team Communication Protocol

Agents express delegation through bracketed response tags. A direct tag such as *[@reviewer: inspect the patch]* creates work for one recipient; multiple recipients enable fan-out. A room tag such as *[#dev: review complete]* posts to the shared team context. The parser counts bracket depth so payloads containing nested array expressions are less likely to be truncated.

Each internal message carries the originating agent, team, thread, root-message identifier, and depth. Representing delegation as another queue item allows the existing persistence and failure machinery to apply to internal work. A final-notification form records a message without invoking its recipient, providing a termination mechanism for exchanges that need acknowledgement without another reasoning turn.

C. Bounding Coordination Cost

Depth limits and per-agent message-rate limits constrain runaway dialogue. Turn and idle timeouts address unresponsive processes. These controls bound specific aspects of activity, but a depth cap alone does not imply a small total conversation. If every invocation creates at most b child messages and the root has depth zero, a tree with maximum depth d contains at most:

$$N(d, b) \leq \sum_{k=0}^{d} b^k. \quad (2)$$

Equation (2) is an analytical bound under the stated fan-out assumption, not a measured property of the implementation. At b = 2 and d = 4, the bound is 31 invocations including the root. Even shallow conversations can therefore amplify provider use. A per-root budget, duplicate-message suppression, and explicit completion criteria would complement the documented depth and rate controls.

D. Memory, Scheduling, and Proactive Work

Persistent memory uses Markdown files with names and summaries in frontmatter. A bounded index is inserted into the prompt, and an agent can open complete files when needed. This separates discoverability from full context loading. It may reduce prompt size compared with including every memory file, but relevance and token savings must be measured against a specific corpus. A provider session identifier supplies conversational continuity; it is not a replacement for durable project knowledge.

Skills package reusable instructions and supporting resources. Identity material defines an agent persona, while workspace instructions establish behavioral guidance. Because these resources influence future tool use, they must be treated as configuration with an associated trust boundary. Persistent storage also makes stale or incorrect guidance durable, so revision and provenance are important operational concerns.

The scheduler enqueues prompts using cron expressions or a one-time execution timestamp. A heartbeat periodically sends an agent its heartbeat instructions, skipping agents that already have pending work. Both paths reuse the normal queue rather than adding a separate execution engine. Nevertheless, skipping a busy agent is a best-effort load policy; it does not establish fairness, schedule precision, or guaranteed completion by a deadline.

A useful memory evaluation should include both stable project facts and deliberately outdated entries. The agent should retrieve relevant information while avoiding unsupported reliance on obsolete instructions. Measuring retrieval correctness and unnecessary file reads would reveal whether the summary index actually helps. Such a study would also distinguish an infrastructure feature, persistent storage, from an outcome, correct recall.

Together, memory and scheduled work extend the service beyond interactive chat. The resulting autonomy remains conditional on host uptime, channel connectivity, valid credentials, provider availability, and the content of the configured prompts.

V. RELIABILITY AND SECURITY ANALYSIS

A. Delivery Semantics and Failure Recovery

Persisting a request before invocation protects accepted queue records from ordinary daemon restarts, subject to successful commits and functioning storage. Startup recovery resets stale processing records and interrupts abandoned run records. Retry counters and a dead-letter queue expose work that cannot complete automatically. Channel supervision and graceful shutdown address additional process-lifecycle failures.

Outgoing responses remain pending until a channel client acknowledges delivery. This supports an at-least-once delivery strategy, but duplicates remain possible. A client can deliver a reply and fail before recording its acknowledgement, causing redelivery. Likewise, an agent may modify a file or call an external tool and then fail before its queue completion is committed. Retrying the input may repeat that side effect.

TABLE I
FAILURE WINDOWS AND THEIR DESIGN IMPLICATIONS

Failure window	**Recovery implication**
Before queue commit	Acceptance is not durable.
After claim; before result	Stale work can be retried.
After external side effect	Retry may repeat the effect.
After send; before ACK	Reply may be delivered again.

Table I distinguishes durable intent from exactly-once external effects. Stable operation identifiers, idempotent actions, and channel-level deduplication are appropriate extensions where duplicates matter. The supplied documentation does not establish that these extensions are implemented. Queue persistence is therefore a recovery mechanism rather than proof that every real-world action executes once.

Recovery policy must also account for slow processes that remain alive. If a legitimate long run is mistakenly classified as stale, a replacement invocation may overlap the original. Conversely, an overly generous threshold delays recovery from actual failure. The documentation names recovery and timeout mechanisms without establishing their behavior at every timing boundary. A useful validation should therefore distinguish a cleanly terminated provider, a hung process, and a delayed but healthy one, and should verify that ownership is reconciled before replacement work begins.

B. Access and Execution Boundaries

The documented access controls include a random bearer token, timing-safe token comparison, origin restrictions, sender pairing, identifier validation, secret redaction, and canonical-path checks for file attachments. Loopback port publishing limits direct remote exposure under the stated Docker configuration. Each control addresses a distinct boundary; origin checks alone do not authenticate non-browser clients.

Attachment path validation resolves real paths and checks containment within allowed roots, addressing traversal and symlink escape in that access path. It does not demonstrate that a child harness has operating-system-level isolation. Shell permissions, inherited environment variables, filesystem mounts, and provider tool policies determine the actual authority of an executing agent. The documented unrestricted-execution setting makes this distinction particularly significant.

Channel pairing restricts which senders can trigger work, but authorized content may still include malicious instructions. Tool output, retrieved documents, and peer-agent messages can also carry prompt injection. Separation of data from instructions, constrained credentials, and review of consequential actions are therefore important design considerations. Their effectiveness requires adversarial testing rather than inference from token authentication.

VI. EVALUATION METHODOLOGY

A. Evidence and Reproducible Setup

The available evidence is the project documentation [2]. It names backend tests using the Node.js test runner and a fake provider fixture, frontend tests with jsdom, static type checking, and linting. No repository execution, test-pass count, or runtime observation was supplied for this paper. The following protocol is proposed to validate documented behavior and establish performance evidence.

A reproducible evaluation should record the source revision, host CPU and memory, operating system, runtime and dependency versions, database settings, provider and model configuration, and channel setup. Separate deterministic fixture runs from live-provider runs. Fixture runs isolate orchestration correctness; live runs characterize end-to-end behavior affected by model latency, usage limits, and stochastic responses.

B. Workloads and Fault Injection

Use a fixed suite covering direct messages, leader-routed team requests, nested-tag parsing, fan-out and final notifications, scheduled prompts, heartbeat suppression, and file delivery. Vary the number of configured agents and the arrival rate while keeping task inputs fixed. Record accepted message identifiers and terminal states, then compare them with provider invocations and channel deliveries.

For security cases, include invalid tokens, unpaired senders, traversal attempts, symlinked attachments, and hostile text in retrieved content. Verify both the rejection behavior and the absence of unauthorized provider invocations.

Inject failures after queue insertion, during harness execution, after a side effect but before completion, and after response delivery but before acknowledgement. Restart the daemon and inspect pending, completed, and dead-letter records. Each test should specify its interruption point and expected invariant, making recovery failures distinguishable from expected duplicate delivery.

C. Metrics and Baselines

Capture enqueue, claim, provider-start, provider-end, and delivery timestamps. Report queue waiting time and end-to-end latency using medians and 95th percentiles, alongside completed root requests per unit time. Also report retries, dead-letter frequency, duplicate deliveries, restart recovery time, peak memory, and database growth. Count internal invocations per root request to expose coordination amplification.

TABLE II
PROPOSED VALIDATION MATRIX; NO MEASURED RESULTS

Test area	Observable criterion
Routing / context	Correct agent, thread, and root.
Serialization	At most one active run per agent.
Restart recovery	Accepted IDs remain traceable.
Loop controls	Depth, rate, and time limits apply.
Access boundaries	Unauthorized requests are rejected.

Compare one agent with a leader-and-reviewer team on the same task set, fixed provider settings, and explicit cost limits. Assess task success using predefined acceptance criteria and independently reviewed artifacts. Report provider token usage and elapsed time together with quality, since additional agents can improve review coverage while also increasing cost or latency. A second baseline with persistence disabled can isolate the contribution of durable queueing under restarts.

Repeat trials with recorded randomization and report sample counts and uncertainty. Separate completed roots from completed internal messages so fan-out does not inflate apparent throughput. Publish sanitized traces and failure scripts where possible. These procedures would support empirical conclusions; the present paper makes none about measured superiority.

Queue growth offers an additional load indicator. If accepted arrivals exceed the sustainable completion rate for a busy agent, its waiting backlog grows despite available workers elsewhere. A balanced aggregate throughput figure can hide this hotspot. Trials should therefore include skewed workloads aimed at one agent as well as evenly distributed requests. Reporting queue length over time, by agent, would reveal whether latency remains bounded at the tested arrival rate and whether scheduled or heartbeat work waits behind long interactive sessions. These are proposed observations, not capacity guarantees.

VII. DISCUSSION AND FUTURE WORK

The architecture offers a practical integration point for collaboration, transport, persistence, and monitoring. Its strongest analytical property is that user input, delegation, scheduled work, and recovery share a common queue abstraction. This reduces the number of distinct execution paths an operator must understand. However, common infrastructure also creates shared bottlenecks and concentrates failures.

Single-host SQLite deployment limits horizontal scaling, and external harness changes can break adapter assumptions. Additional channel integrations increase credential and session-management demands; the documented WhatsApp integration depends on an unofficial web client. Shared bearer-token access is also less suitable for multiple independent users than role-based authorization.

Future work should prioritize idempotency and operation provenance, per-root execution budgets, authenticated SSE integration tests, memory revision policies, and permission isolation. Multi-node execution would require explicit worker ownership, distributed coordination, and a storage migration strategy. Such a redesign should follow measured bottlenecks rather than assuming that a different database alone provides distributed correctness.

Generalization is limited by the evidence source and deployment assumptions. Documentation can describe intended behavior without proving that every path matches that intent. Provider behavior can change independently of the orchestration layer, and a successful controlled fixture does not establish success on open-ended user tasks. Future empirical work should preserve this separation by reporting infrastructure correctness, application task quality, and user experience as distinct outcomes. Doing so would make negative findings actionable: a delivery defect calls for transport repair, whereas an incorrect artifact may require better prompts, tools, or review criteria.

VIII. CONCLUSION

AgentOps integrates provider-backed agent teams with messaging channels and a web dashboard through a durable, event-driven backend. The design combines per-agent serialized execution, message-based delegation, persistent context, scheduling, and inspectable recovery states. Analysis shows how these mechanisms support recoverable work while leaving duplicate effects, coordination amplification, and host-level availability unresolved. The proposed evaluation protocol defines the evidence needed to test reliability, task quality, latency, and resource use. AgentOps is therefore presented as a documented system architecture and research prototype, with empirical validation remaining future work.

REFERENCES

[1] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," arXiv:2210.03629, 2022. doi: 10.48550/arXiv.2210.03629.

[2] "AgentOps - Complete project documentation," unpublished project documentation, supplied as PROJECT_DOCUMENTATION.md, n.d.

[3] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023. doi: 10.48550/arXiv.2308.08155.

[4] S. Hong *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," arXiv:2308.00352, 2023. doi: 10.48550/arXiv.2308.00352.

[5] SQLite, "Write-ahead logging." Accessed: Oct. 17, 2026. [Online]. Available: https://sqlite.org/wal.html

[6] WHATWG, "Server-sent events," *HTML Living Standard*. Accessed: Oct. 17, 2026. [Online]. Available: https://html.spec.whatwg.org/multipage/server-sent-events.html

ANNEXURE B

PLAGIARISM REPORT

The final institutional plagiarism or similarity report is to be inserted in this annexure after the approved checking process. A similarity report must identify the submitted document and the checking record so that the result can be associated with the version reviewed.

No similarity percentage or clearance is asserted on this page. The original output from the institutional service and any required guide verification belong here. If revisions are made after checking the report should be regenerated or verified under the department's procedure.

Verification field	**Record**
Report title	CLOUD-BASED MULTI-AGENT AI ORCHESTRATION PLATFORM
Team and group	Cloud Clash BCCC41
Checking service	To be completed by the institution
Submission or report identifier	_____
Date checked	_____
Similarity result	Attach authentic generated report
Guide verification	_____

SPACE FOR THE AUTHENTIC PLAGIARISM REPORT

ANNEXURE C

EMPATHY CHART AND SOLUTION DOCUMENT

The following empathy chart is a proposed design exercise for the intended platform operator. It organizes assumptions that should be checked through user discussion and project demonstrations. It is not presented as the result of conducted interviews or a user study.

Perspective	**Proposed user need or concern**
Says	I want to send a task through a familiar interface and know whether it finished.
Thinks	I need to trust that pending work will remain visible after a restart.
Does	Switches between chat tools project files and task boards to coordinate work.
Feels	Uncertain when an assistant is busy but its current state is unclear.
Pain	Repeated context manual handoffs disconnected tools and ambiguous failures.
Gain	Traceable tasks clear ownership recoverable execution and inspectable outcomes.

Validation should ask users to follow one request across routing execution and delivery. Record where the interface leaves them uncertain and whether the displayed state changes their next action. Revise this chart when observation contradicts the proposed assumptions.

SOLUTION DOCUMENT

The solution maps user concerns to concrete platform mechanisms. Each mechanism must be evaluated at its own boundary. For example durable queueing helps preserve accepted work but does not guarantee that an external action was performed only once. Clear interface wording should reflect those limits.

User concern	**Proposed mechanism**	**Validation question**
Familiar access	Messaging clients and dashboard	Can an approved user submit and receive work
Work ownership	Agent and team routing	Is the selected agent understandable
Lost progress	Queue state and recovery	Can accepted IDs be reconciled after restart
Repeated context	Thread sessions and memory	Is relevant context recovered correctly
Hidden failure	Dead-letter and run inspection	Can the operator identify a next action
Unbounded team activity	Depth rate and timeout controls	Does work stop at the configured limits
Unclear completion	Artifact acceptance criteria	Does the result satisfy the original request

The first user review should focus on a small set of realistic tasks rather than a broad satisfaction claim. Keep task inputs observations and suggested changes with the project tracker so that interface decisions remain connected to evidence.

ANNEXURE D

PROJECT TRACKER

This tracker provides a review structure for the registered team. Actual dates ownership and completion status should be entered from project records. The project documentation describes capabilities but does not supply a signed milestone history. The entries therefore identify evidence still to be recorded for verification.

Milestone	**Evidence to attach**	**Review status**
Scope and title	Approved portal record and requirements	Title supplied in portal image
Architecture	Versioned component and state diagrams	Review with guide
Queue and adapters	Fixture logs linked to source revision	Verification pending
Team routing	Root and child message trace	Verification pending
Channels	Pairing send and acknowledgement evidence	Verification pending
Dashboard	Connected demonstration and state checks	Verification pending
Recovery and security	Fault and access test outputs	Verification pending
Evaluation	Workload results and reviewed artifacts	Verification pending
Submission	Final report paper and similarity output	Verification pending

TEAM AND REVIEW RECORD

Project: CLOUD-BASED MULTI-AGENT AI ORCHESTRATION PLATFORM

Team: Cloud Clash

Group: BCCC41

Guide: Prof. Vrushali Kondhalkar

Academic year: 2026–27

Team member	**Enrollment number**	**Assigned work reference**
Tanishq Sachin Sonawane	ADT24SOCBD132	To be recorded in team log
Keyur Arvind Mhatre	ADT24SOCBD062	To be recorded in team log
Varun Hitendra Patil	ADT24SOCBD139	To be recorded in team log

Guide review record

Review item	**Record**
Review date	_____
Software revision	_____
Evidence reviewed	_____
Actions and owner	_____
Next review date	_____
Guide signature	_____

Keep implementation changes demonstration evidence and report revisions synchronized. When a test fails record the observed behavior and the corrective action rather than overwriting the earlier result.